

자바 개발자를 위한

JavaScript 입문

김아람 지음

자바 개발자를 위한 JavaScript 입문

지은이 김아람

초판 1쇄 발행 2026년 9월 15일

펴낸곳 부크크(Bookk)

주소 서울특별시 강남구 테헤란로 142

전화 1670-8316

이메일 info@bookk.co.kr

홈페이지 www.bookk.co.kr

ISBN 979-11-XXXX-XXX-X (93000)

이 책의 저작권은 저자에게 있습니다.

저작권법에 의해 보호받는 저작물이므로 무단 전재와 복제를 금합니다.

이 책 내용의 전부 또는 일부를 이용하려면 반드시 저작권자의 서면 동의를 받아야 합니다.

잘못된 책은 구입하신 곳에서 교환해 드립니다.

목 차

0장	오리엔테이션	9
1장	JavaScript 첫걸음	21
2장	변수와 타입	35
3장	함수의 모든 것	55
4장	객체와 배열	77
5장	클래스와 모듈	101
6장	비동기 프로그래밍	125
7장	DOM과 이벤트	149
8장	모던 JavaScript 생태계	175
9장	실전 프로젝트	197

오리엔테이션

이 책의 대상과 목표, Java 개발자가 JavaScript를 배울 때 알아야 할 핵심 차이점, 그리고 학습 환경을 설정합니다.

이 책의 대상

이 책은 Java 백엔드 개발자를 위해 쓰였습니다. Spring Boot로 REST API를 만들어 본 경험이 있고, 이제 프론트엔드 영역을 이해하거나 직접 개발하고 싶은 분들을 대상으로 합니다.

다음과 같은 분들에게 이 책이 도움이 됩니다:

JavaScript를 처음 배우는 Java 개발자
JavaScript를 사용해봤지만 체계적으로 정리하고 싶은 분
React, Vue, Angular를 배우기 전 기초를 다지고 싶은 분
Node.js 백엔드를 시작하려는 분
풀스택 개발자로 성장하고 싶은 분

왜 Java 개발자 관점인가?

JavaScript 입문서는 많습니다. 그러나 대부분 프로그래밍을 처음 배우는 사람을 대상으로 합니다. 변수가 무엇인지, 반복문이 무엇인지부터 설명합니다.

Java 개발자는 이미 프로그래밍의 기본 개념을 알고 있습니다. 필요한 것은 "Java와 무엇이 다른가"입니다.

Java 개발자 포인트

이 책은 Java 문법 및 개념과 비교하며 JavaScript를 설명합니다. 이미 알고 있는 지식을 활용해 새로운 언어를 빠르게 익힐 수 있습니다.

Java와 JavaScript의 근본적 차이

이름이 비슷해서 관련 있어 보이지만, Java와 JavaScript는 완전히 다른 언어입니다. 시작하기 전에 가장 중요한 차이점 5가지를 먼저 짚고 갑니다.

1. 타입 시스템

JAVA — 정적 타입

```
String name = "홍길동";  
int age = 30;  
// age = "서른"; // 컴파일 에러!
```

JAVASCRIPT — 동적 타입

```
let name = "홍길동";  
let age = 30;  
age = "서른"; // 문제 없음!
```

JavaScript는 변수에 타입을 지정하지 않습니다. 같은 변수에 숫자를 넣었다가 문자열을 넣을 수도 있습니다. 자유롭지만, 실수하기도 쉽습니다.

2. 클래스 vs 프로토타입

JAVA — 클래스 기반

```
public class Person {  
    private String name;  
  
    public Person(String name) {  
        this.name = name;  
    }  
}
```

JAVASCRIPT — 프로토타입 기반

```
function Person(name) {
  this.name = name;
}
// 또는 ES6 class 문법
class Person {
  constructor(name) {
    this.name = name;
  }
}
```

JavaScript도 ES6부터 `class` 문법을 지원하지만, 내부적으로는 프로토타입 기반입니다. 5장에서 자세히 다룹니다.

3. 함수는 일급 객체

JAVA — 메서드는 객체가 아님

```
// Java 8+ 람다로 비슷하게 가능
Function<Integer, Integer> square =
    x -> x * x;

int result = square.apply(5);
```

JAVASCRIPT — 함수도 값

```
const square = function(x) {
  return x * x;
};

// 또는 화살표 함수
const square = x => x * x;

const result = square(5);
```

JavaScript에서 함수는 변수에 담을 수 있고, 다른 함수의 인자로 전달할 수 있으며, 함수에서 함수를 반환할 수도 있습니다. 이것이 JavaScript의 핵심입니다.

4. 비동기 처리

JAVA — 스레드 기반

```
CompletableFuture.supplyAsync(() -> {
    return fetchData();
}).thenAccept(data -> {
    process(data);
});
```

JAVASCRIPT — 이벤트 루프

```
// Promise
fetchData()
    .then(data => process(data));

// async/await
const data = await fetchData();
process(data);
```

JavaScript는 단일 스레드에서 이벤트 루프를 통해 비동기를 처리합니다. 멀티스레딩 대신 콜백, Promise, async/await를 사용합니다. 6장에서 깊이 다룹니다.

5. 실행 환경

JAVA

```
// JVM 위에서 실행
// 컴파일 필요 (javac)
// 어디서나 같은 환경
```

JAVASCRIPT

```
// 브라우저 또는 Node.js에서 실행
// 인터프리터 방식 (JIT 컴파일)
// 환경마다 API가 다름
```

JavaScript는 브라우저에서 실행하면 `document`, `window` 같은 DOM API를 쓸 수 있고, Node.js에서 실행하면 `fs`, `http` 같은 서버 API를 쓸 수 있습니다.

학습 환경 설정

JavaScript를 실행하는 두 가지 방법을 준비합니다.

1. 브라우저 개발자 도구

가장 빠른 방법입니다. Chrome, Firefox, Edge 등 모던 브라우저에서 **F12** 또는 **Cmd+Option+I**를 눌러 개발자 도구를 열고, Console 탭에서 바로 JavaScript를 실행할 수 있습니다.

```
// 브라우저 콘솔에서 실행해보세요
console.log("Hello, JavaScript!");
alert("환영합니다!");
```

2. Node.js 설치

서버 사이드 JavaScript를 실행하려면 Node.js가 필요합니다. nodejs.org에서 LTS 버전을 다운로드하세요.

```
// 설치 확인
node --version    // v18.x.x 또는 그 이상
npm --version     // 9.x.x 또는 그 이상
```

터미널에서 **node** 를 입력하면 REPL(대화형 셸)이 시작됩니다:

```
$ node
> console.log("Hello from Node.js!")
Hello from Node.js!
> 2 + 3
5
> .exit
```

3. VS Code 설정 (권장)

Visual Studio Code는 JavaScript 개발에 가장 많이 사용되는 에디터입니다. 다음 확장을 설치하면 편리합니다:

ESLint — 코드 품질 검사

Prettier — 코드 포매팅

JavaScript (ES6) code snippets — 코드 자동완성

이 책의 학습 방법

비교하며 읽기: 각 개념을 Java와 비교합니다. 익숙한 것에서 출발하세요.

직접 실행하기: 예제 코드를 브라우저 콘솔이나 Node.js에서 직접 실행해보세요.

변형해보기: 예제를 조금씩 바꿔보면서 동작을 확인하세요.

에러 경험하기: 일부러 에러를 내보고 메시지를 읽어보세요.

준비 완료!

다음 장부터 본격적으로 JavaScript를 배웁니다. 1장에서는 JavaScript의 역사와 특징, 그리고 첫 번째 프로그램을 작성합니다.

JavaScript 첫걸음

JavaScript의 역사와 특징을 이해하고, Java와의 근본적인 차이를 파악합니다. 그리고 첫 번째 프로그램을 작성합니다.

JavaScript의 탄생

1995년, Netscape의 브렌던 아이크(Brendan Eich)가 단 10일 만에 만든 언어입니다. 당시 이름은 Mocha였다가 LiveScript로, 그리고 마케팅 목적으로 JavaScript가 되었습니다.

이름의 함정

JavaScript와 Java는 이름만 비슷할 뿐, 완전히 다른 언어입니다. "Java"를 붙인 건 당시 Java의 인기를 이용하려는 마케팅 전략이었습니다.

처음에는 브라우저에서 간단한 폼 검증용으로 만들어졌지만, 지금은 프론트엔드, 백엔드, 모바일, 데스크톱 앱까지 모든 곳에서 사용됩니다.

JavaScript의 발전

연도	버전	주요 기능
1997	ES1	첫 표준화 (ECMAScript)
2009	ES5	strict mode, JSON, Array 메서드
2015	ES6 (ES2015)	let/const, 화살표 함수, 클래스, Promise, 모듈
2016~	ES2016+	async/await, 옵셔널 체이닝 등 매년 업데이트

ES6(2015)가 가장 큰 변화였습니다. 현대 JavaScript는 대부분 ES6 이후 문법을 사용합니다. 이 책도 ES6+ 문법을 기준으로 설명합니다.

Java와 JavaScript 비교

구분	Java	JavaScript
타입	정적 타입 (컴파일 시 검사)	동적 타입 (런타임에 결정)
실행	JVM (컴파일 필요)	브라우저/Node.js (인터프리터)
객체	클래스 기반	프로토타입 기반
함수	메서드 (클래스에 종속)	일급 객체 (독립적)
스레드	멀티스레드	싱글스레드 + 이벤트 루프
세미콜론 필수		선택 (ASD)

Hello, JavaScript!

이제 첫 번째 프로그램을 작성해봅시다.

방법 1: 브라우저 콘솔

브라우저에서 **F12**를 눌러 개발자 도구를 열고, Console 탭에 입력하세요:

```
console.log("Hello, JavaScript!");

// 변수 선언
let name = "Java 개발자";
console.log("안녕하세요, " + name + "님!");

// 템플릿 리터럴 (ES6)
console.log(`안녕하세요, ${name}님!`);
```

방법 2: HTML 파일

`hello.html` 파일을 만들어보세요:

```

<!DOCTYPE html>
<html>
<head>
  <title>Hello JavaScript</title>
</head>
<body>
  <h1 id="greeting"></h1>

  <script>
    const name = "Java 개발자";
    document.getElementById("greeting").textContent =
      `안녕하세요, ${name}님!`;
  </script>
</body>
</html>

```

방법 3: Node.js

`hello.js` 파일을 만들고 터미널에서 실행하세요:

```

// hello.js
const name = "Java 개발자";
console.log(`안녕하세요, ${name}님!`);

// 실행: node hello.js

```

Java vs JavaScript 문법 비교

변수 선언

JAVA

```

String message = "Hello";
final int MAX = 100;
int count = 0;

```

JAVASCRIPT

```
let message = "Hello";
const MAX = 100;
var count = 0; // 옛날 방식
```

JavaScript에서는 `let` (변경 가능), `const` (상수), `var` (구버전)를 사용합니다. 타입은 명시하지 않습니다.

출력

JAVA

```
System.out.println("Hello");
System.out.printf("Count: %d", 10);
```

JAVASCRIPT

```
console.log("Hello");
console.log("Count:", 10);
console.log(`Count: ${10}`);
```

문자열 연결

JAVA

```
String name = "홍길동";
String msg = "안녕, " + name + "!";

// Java 15+ 텍스트 블록
String html = """
    <html>
        <body>%s</body>
    </html>
    """.formatted(name);
```

JAVASCRIPT

```
const name = "홍길동";
const msg = `안녕, ${name}!`;

// 템플릿 리터럴 (백틱)
const html = `
  <html>
    <body>${name}</body>
  </html>
`;
```

템플릿 리터럴

JavaScript의 백틱(```)은 Java의 텍스트 블록과 비슷하지만, `${}` 안에 어떤 표현식이든 넣을 수 있어 더 강력합니다.

세미콜론은 필수인가?

JavaScript는 세미콜론을 생략할 수 있습니다. 엔진이 자동으로 삽입해주기 때문입니다(Automatic Semicolon Insertion, ASI).

```
// 둘 다 유효
let a = 1;
let b = 2

console.log(a + b)
```

하지만 가끔 예상치 못한 동작이 발생할 수 있어, 많은 팀에서는 세미콜론을 명시적으로 작성하는 규칙을 사용합니다. ESLint와 Prettier로 일관성을 유지하는 것이 좋습니다.

strict mode

ES5에서 도입된 엄격 모드입니다. 파일 맨 위에 다음을 추가하면 활성화됩니다:

```
"use strict";

// 이제 선언 없이 변수 사용 불가
x = 10; // ReferenceError!

// 반드시 선언 필요
let x = 10; // OK
```

ES6 모듈(`import` / `export`)을 사용하면 자동으로 strict mode가 적용됩니다. Java 개발자에게 익숙한 엄격한 환경을 원한다면 strict mode를 사용하세요.

정리

JavaScript는 Java와 이름만 비슷한 완전히 다른 언어
동적 타입, 프로토타입 기반, 함수가 일급 객체
ES6(2015)가 현대 JavaScript의 기준
브라우저 콘솔, HTML 파일, Node.js에서 실행 가능
`let` / `const` 사용, 템플릿 리터럴(``${}``) 활용

변수와 타입

Java의 정적 타입과 달리 JavaScript는 동적 타입 언어입니다. var, let, const의 차이를 이해하고, JavaScript의 타입 시스템을 파악합니다.

var, let, const

JavaScript에서 변수를 선언하는 방법은 세 가지입니다. Java에서는 타입을 명시하지만, JavaScript에서는 키워드로 변수의 특성을 결정합니다.

JAVA

```
// 변경 가능
String name = "홍길동";
name = "김철수";

// 상수
final int MAX = 100;
// MAX = 200; // 컴파일 에러
```

JAVASCRIPT

```
// 변경 가능 (권장)
let name = "홍길동";
name = "김철수";

// 상수
const MAX = 100;
// MAX = 200; // TypeError
```

var - 구시대의 유물

`var`는 ES6 이전에 사용하던 변수 선언 방식입니다. 함수 스코프를 가지며, 호이스팅이라는 독특한 동작을 합니다.

```
// var의 문제점 1: 함수 스코프
function example() {
  if (true) {
    var x = 10;
  }
  console.log(x); // 10 - 블록 밖에서도 접근 가능!
}

// var의 문제점 2: 호이스팅
console.log(y); // undefined (에러가 아님!)
var y = 20;

// 위 코드는 내부적으로 이렇게 해석됨
// var y;           // 선언이 맨 위로 끌어올려짐
// console.log(y); // undefined
// y = 20;
```

var 사용 금지

새로운 코드에서 `var`를 사용하면 안 됩니다. `let` 과 `const` 만 사용하세요. 레거시 코드에서 `var`를 보면 리팩토링 대상입니다.

let - 재할당 가능한 변수

`let`은 ES6에서 도입된 블록 스코프 변수입니다. Java의 일반 변수와 유사하게 동작합니다.

```
// 블록 스코프
if (true) {
  let x = 10;
  console.log(x); // 10
}
// console.log(x); // ReferenceError - 블록 밖에서 접근 불가

// 호이스팅은 되지만 TDZ(Temporal Dead Zone) 적용
// console.log(y); // ReferenceError
let y = 20;

// 재할당 가능
let count = 0;
count = 1;
count = 2;
```

const - 상수

`const` 는 재할당이 불가능한 상수입니다. Java의 `final` 과 비슷하지만 중요한 차이가 있습니다.

```
const PI = 3.14159;
// PI = 3.14; // TypeError: Assignment to constant variable

// 반드시 선언과 동시에 초기화
// const X; // SyntaxError
const X = 100;
```

const의 의미

`const` 는 "값이 변하지 않는다"가 아니라 "재할당이 불가능하다"입니다. 객체나 배열의 내용은 변경할 수 있습니다.

```
const user = { name: "홍길동" };
user.name = "김철수"; // OK! 객체 내용 변경은 가능
console.log(user);    // { name: "김철수" }

// user = { name: "이영희" }; // TypeError: 재할당 불가

const numbers = [1, 2, 3];
numbers.push(4);      // OK! 배열 내용 변경은 가능
console.log(numbers); // [1, 2, 3, 4]
```

어떤 것을 사용해야 할까?

상황	사용	예시
기본값	<code>const</code>	<code>const API_URL = "..."</code>
재할당 필요	<code>let</code>	<code>let count = 0; count++;</code>
절대 사용 금지	<code>var</code>	레거시 코드에서만 발견

기본적으로 `const` 를 사용하고, 재할당이 필요한 경우에만 `let` 을 사용하세요. 이렇게 하면 의도치 않은 값 변경을 방지할 수 있습니다.

동적 타입 vs 정적 타입

Java 개발자에게 가장 어색한 부분입니다. JavaScript는 변수에 타입을 지정하지 않으며, 런타임에 타입이 결정됩니다.

JAVA

```
// 컴파일 시 타입 확정
String name = "홍길동";
// name = 123; // 컴파일 에러

int count = 10;
// count = "ten"; // 컴파일 에러

// 타입 변환은 명시적으로
int num = Integer.parseInt("123");
```

JAVASCRIPT

```
// 런타임에 타입 결정
let name = "홍길동";
name = 123; // OK! 문자열 → 숫자

let count = 10;
count = "ten"; // OK! 숫자 → 문자열

// 타입 변환은 자동으로 (암묵적 변환)
let num = "123" * 1; // 123 (숫자)
```

동적 타입의 장단점

장점: 유연하고 빠른 개발, 프로토타이핑에 유리

단점: 런타임 에러 가능성, IDE 자동완성 제한, 대규모 프로젝트에서 유지보수 어려움

→ 이런 단점을 보완하기 위해 TypeScript가 등장했습니다 (8장에서 소개)

JavaScript의 타입들

JavaScript에는 7개의 원시 타입(Primitive)과 1개의 객체 타입(Object)이 있습니다.

원시 타입 (Primitive Types)

타입	설명	예시	Java 대응
<code>string</code>	문자열	"hello", 'world', `템플릿`String	
<code>number</code>	숫자 (정수/실수 구분 없음)	42, 3.14, NaN, Infinity	int, double 통합
<code>bigint</code>	큰 정수	9007199254740993n	BigInteger
<code>boolean</code>	불리언	true, false	boolean
<code>undefined</code>	값이 할당되지 않음	undefined	없음
<code>null</code>	의도적으로 비어있음	null	null
<code>symbol</code>	고유 식별자	Symbol("id")	없음

number - 모든 숫자를 하나로

Java에서는 `int`, `long`, `float`, `double` 등 여러 숫자 타입이 있지만, JavaScript는 모두 `number` 하나입니다.

```
let integer = 42;           // 정수
let float = 3.14;          // 실수
let negative = -100;       // 음수
let hex = 0xFF;            // 16진수 (255)
let binary = 0b1010;      // 2진수 (10)
let octal = 0o777;        // 8진수 (511)

// 특수한 숫자 값
let inf = Infinity;       // 무한대
let negInf = -Infinity;   // 음의 무한대
let notNum = NaN;         // Not a Number

// 주의: 부동소수점 오차
console.log(0.1 + 0.2);    // 0.30000000000000004
console.log(0.1 + 0.2 === 0.3); // false!
```

부동소수점 주의

JavaScript의 숫자는 IEEE 754 배정밀도 부동소수점입니다. 금융 계산처럼 정밀도가 중요한 경우 정수로 변환하거나 라이브러리를 사용하세요.

undefined vs null

Java에는 `null` 만 있지만, JavaScript에는 `undefined` 도 있습니다. 둘의 차이를 이해하는 것이 중요합니다.

```
// undefined: 값이 할당되지 않은 상태
let a;
console.log(a); // undefined

function foo(x) {
  console.log(x); // 인자 없이 호출하면 undefined
}
foo();

// null: 의도적으로 "비어있음"을 표현
let user = null; // 아직 사용자가 없음

// 타입 확인 (주의!)
typeof undefined // "undefined"
typeof null      // "object" (JavaScript의 역사적 버그)

// 비교
undefined == null // true (동등 비교)
undefined === null // false (일치 비교)
```

undefined vs null 사용 가이드

`undefined` 는 시스템이 자동으로 부여하는 "값 없음"

`null` 은 개발자가 명시적으로 부여하는 "빈 값"

변수를 비우고 싶을 때는 `null` 을 사용하세요.

객체 타입 (Object)

원시 타입을 제외한 모든 것은 객체입니다. 배열, 함수도 객체입니다.

```

// 객체 리터럴
const user = {
  name: "홍길동",
  age: 30
};

// 배열 (Array도 객체)
const numbers = [1, 2, 3];

// 함수 (Function도 객체)
function greet() {
  return "Hello";
}

// 타입 확인
typeof user      // "object"
typeof numbers   // "object" (배열도 object)
typeof greet     // "function" (특별 취급)

// 배열 확인은 Array.isArray 사용
Array.isArray(numbers) // true
Array.isArray(user)    // false

```

typeof 연산자

런타임에 타입을 확인하는 연산자입니다. Java의 `instanceof` 와 비슷하지만 원시 타입에 사용됩니다.

```

typeof "hello"    // "string"
typeof 42         // "number"
typeof true       // "boolean"
typeof undefined  // "undefined"
typeof Symbol()   // "symbol"
typeof 10n        // "bigint"

// 주의해야 할 케이스
typeof null       // "object" (버그!)
typeof []         // "object"
typeof {}         // "object"
typeof function(){} // "function"

```

```
// instanceof로 타입 확인
if (obj instanceof String) {
    String s = (String) obj;
}

// Java 16+ 패턴 매칭
if (obj instanceof String s) {
    System.out.println(s.length());
}
```

JAVASCRIPT

```
// typeof로 원시 타입 확인
if (typeof value === "string") {
    console.log(value.length);
}

// instanceof로 객체 타입 확인
if (obj instanceof Array) {
    console.log(obj.length);
}
```

타입 변환

JavaScript는 자동으로 타입을 변환합니다(암묵적 변환). Java 개발자에게는 이 부분이 가장 혼란스러울 수 있습니다.

암묵적 변환 (Type Coercion)

```
// 문자열 연결 - 숫자가 문자열로 변환
"3" + 2          // "32" (문자열)
2 + "3"          // "23" (문자열)

// 산술 연산 - 문자열이 숫자로 변환
"6" - 2          // 4 (숫자)
"6" * "2"        // 12 (숫자)
"10" / 2         // 5 (숫자)

// 비교 연산
"5" == 5         // true (타입 변환 후 비교)
"5" === 5        // false (타입까지 비교)

// 논리 연산에서의 변환 (Truthy/Falsy)
if ("hello") { } // true - 비어있지 않은 문자열
if (0) { }       // false - 0은 falsy
if ([]) { }      // true - 빈 배열도 truthy!
```

+ 연산자 주의

+ 는 문자열 연결과 숫자 덧셈 둘 다 사용됩니다. 피연산자 중 하나라도 문자열이면 문자열 연결이 됩니다. 숫자 연산이 필요하면 `Number()` 로 명시적 변환하세요.

명시적 변환

```
// 문자열로 변환
String(123)      // "123"
(123).toString() // "123"
123 + ""         // "123" (암묵적)

// 숫자로 변환
Number("123")    // 123
parseInt("123")  // 123 (정수)
parseFloat("3.14") // 3.14 (실수)
+"123"           // 123 (단항 + 연산자)

// 불리언으로 변환
Boolean(1)       // true
Boolean(0)       // false
Boolean("")      // false
Boolean("hello") // true
!!value          // 불리언으로 변환하는 관용적 표현
```

Falsy 값들

JavaScript에서 조건문에 사용될 때 `false` 로 평가되는 값들입니다.

Falsy 값	설명
<code>false</code>	불리언 false
<code>0</code> , <code>-0</code>	숫자 0
<code>""</code> , <code>' '</code> , <code>` `</code>	빈 문자열
<code>null</code>	null
<code>undefined</code>	undefined
<code>NaN</code>	Not a Number

이 외의 모든 값은 truthy입니다. 빈 배열 `[]`, 빈 객체 `{}` 도 truthy입니다.

```
// Java 개발자 주의!
if ([]) {
    console.log("빈 배열은 truthy!"); // 실행됨
}

if ({}) {
    console.log("빈 객체도 truthy!"); // 실행됨
}

// 배열/객체 비어있는지 확인하는 방법
if (arr.length === 0) { } // 빈 배열
if (Object.keys(obj).length === 0) { } // 빈 객체
```

== vs === (동등 vs 일치)

JavaScript에는 두 가지 비교 연산자가 있습니다. Java 개발자에게 가장 중요한 포인트입니다.

JAVA

```
// == 는 참조 비교
String a = new String("hello");
String b = new String("hello");
a == b // false (다른 객체)
a.equals(b) // true (값 비교)

// 원시 타입은 == 로 값 비교
int x = 5;
int y = 5;
x == y // true
```

JAVASCRIPT

```
// == 는 타입 변환 후 비교 (느슨한 비교)
"5" == 5 // true
0 == false // true
null == undefined // true

// === 는 타입까지 비교 (엄격한 비교)
"5" === 5 // false
0 === false // false
null === undefined // false
```

항상 === 사용하기

== 의 타입 변환 규칙은 복잡하고 예측하기 어렵습니다. 항상 === 를 사용하세요. ESLint의 `eqeqeq` 규칙을 활성화하면 강제할 수 있습니다.

```
// == 의 이상한 동작들
[] == false    // true (둘 다 0으로 변환)
[] == ![]      // true (??????)
"0" == false   // true
"0" === false  // false

// null 체크에서만 == 사용이 유용
if (value == null) {
    // value가 null 또는 undefined일 때
}

// 이것은 아래와 동일
if (value === null || value === undefined) { }
```

Java 개발자가 주의할 점

1. 선언 없이 변수 사용 금지

```
// Java에서는 불가능하지만 JavaScript에서는...
function bad() {
    x = 10; // 전역 변수가 됨! (strict mode에서는 에러)
}

// 항상 let/const 사용
function good() {
    let x = 10;
}
```

2. const 객체의 내용은 변경 가능

```
// Java의 final과 다름
final List<String> list = new ArrayList<>();
// Java에서도 list.add("item") 가능

// JavaScript도 마찬가지
const arr = [];
arr.push("item"); // OK

// 진짜 불변 객체가 필요하다면
const frozen = Object.freeze({ name: "홍길동" });
frozen.name = "김철수"; // 무시됨 (strict mode에서는 에러)
console.log(frozen.name); // "홍길동"
```

3. NaN은 자기 자신과 같지 않다

```
const result = "hello" * 2; // NaN

// NaN 확인 방법
result === NaN // false (NaN은 자기 자신과 같지 않음!)
isNaN(result) // true (하지만 문제가 있음)
isNaN("hello") // true (문자열도 true)
Number.isNaN(result) // true (정확한 방법)
Number.isNaN("hello") // false
```

4. 문자열 비교

```
// Java
String a = "hello";
String b = "hello";
a.equals(b); // true

// JavaScript - 문자열은 원시 타입이라 === 사용
const a = "hello";
const b = "hello";
a === b; // true (객체가 아니므로 === 로 값 비교 가능)
```

정리

`const` 를 기본으로, 필요할 때만 `let`, `var` 는 절대 사용 금지

JavaScript는 동적 타입 - 변수의 타입이 런타임에 결정됨

7개의 원시 타입: string, number, bigint, boolean, undefined, null, symbol

객체 타입: Object (배열, 함수 포함)

`typeof` 로 타입 확인, `null` 은 `"object"` 로 나오는 버그 주의

암묵적 타입 변환 주의, 특히 `+` 연산자

항상 `===` 사용, `==` 는 피하기

Falsy 값: false, 0, "", null, undefined, NaN

함수의 모든 것

JavaScript에서 함수는 단순한 코드 블록이 아닌 일급 객체입니다. 함수 선언 방식, 화살표 함수, 클로저를 이해하고 Java의 메서드와 비교합니다.

Java의 메서드 vs JavaScript의 함수

Java에서 메서드는 반드시 클래스 안에 있어야 합니다. JavaScript에서 함수는 독립적으로 존재할 수 있으며, 변수에 할당하거나 다른 함수에 전달할 수 있습니다.

JAVA

```
// 메서드는 클래스에 종속
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}

// 사용하려면 인스턴스 필요
Calculator calc = new Calculator();
int result = calc.add(1, 2);
```

JAVASCRIPT

```
// 함수는 독립적으로 존재
function add(a, b) {
    return a + b;
}

// 바로 사용 가능
const result = add(1, 2);

// 변수에 할당 가능
const myAdd = add;
myAdd(1, 2); // 3
```

일급 객체 (First-class Object)

JavaScript에서 함수는 일급 객체입니다. 즉:

1. 변수에 할당할 수 있음
2. 다른 함수의 인자로 전달할 수 있음
3. 다른 함수의 반환값이 될 수 있음
4. 배열이나 객체에 저장할 수 있음

함수 선언 방식

JavaScript에서 함수를 정의하는 방법은 여러 가지가 있습니다.

1. 함수 선언문 (Function Declaration)

```
function greet(name) {
    return `안녕하세요, ${name}님!`;
}

console.log(greet("홍길동")); // "안녕하세요, 홍길동님!"
```

함수 선언문은 호이스팅됩니다. 코드의 어디서든 함수를 호출할 수 있습니다.

```
// 선언 전에 호출해도 동작함 (호이스팅)
sayHello(); // "Hello!"

function sayHello() {
  console.log("Hello!");
}
```

2. 함수 표현식 (Function Expression)

```
const greet = function(name) {
  return `안녕하세요, ${name}님!`;
};

console.log(greet("홍길동")); // "안녕하세요, 홍길동님!"
```

함수 표현식은 호이스팅되지 않습니다. 선언 전에 호출하면 에러가 발생합니다.

```
// sayHello(); // ReferenceError: Cannot access before initialization

const sayHello = function() {
  console.log("Hello!");
};

sayHello(); // "Hello!"
```

3. 화살표 함수 (Arrow Function)

ES6에서 도입된 간결한 함수 문법입니다. Java의 람다 표현식과 비슷합니다.

JAVA (람다)

```
// 함수형 인터페이스 필요
Function<String, String> greet =
    name -> "안녕, " + name + "!";

// Stream에서 주로 사용
list.stream()
    .filter(x -> x > 0)
    .map(x -> x * 2)
    .collect(Collectors.toList());
```

JAVASCRIPT

```
// 화살표 함수
const greet = name => `안녕, ${name}!`;

// 배열 메서드에서 주로 사용
const result = list
    .filter(x => x > 0)
    .map(x => x * 2);
```

```
// 다양한 화살표 함수 형태

// 파라미터 1개: 괄호 생략 가능
const square = x => x * x;

// 파라미터 0개: 괄호 필수
const getRandom = () => Math.random();

// 파라미터 2개 이상: 괄호 필수
const add = (a, b) => a + b;

// 본문이 여러 줄: 중괄호와 return 필요
const greet = name => {
    const message = `안녕하세요, ${name}님!`;
    return message;
};

// 객체 반환: 괄호로 감싸기
const createUser = name => ({ name: name, age: 0 });
```

화살표 함수의 this

화살표 함수는 자신만의 `this` 를 가지지 않습니다. 외부 스코프의 `this` 를 그대로 사용합니다. 이 특성은 콜백 함수에서 유용하지만, 메서드 정의에는 부적합합니다.

함수 선언 방식 비교

방식	호이스팅	this	용도
함수 선언문 O	동적 바인딩		일반 함수, 메서드
함수 표현식 X	동적 바인딩		콜백, 조건부 정의
화살표 함수 X	렉시컬 (상위 스코프)		콜백, 짧은 함수

파라미터와 인자

기본값 파라미터

JAVA

```
// 기본값 파라미터 없음
// 메서드 오버로딩으로 대체
public String greet(String name) {
    return greet(name, "안녕");
}

public String greet(String name, String msg) {
    return msg + ", " + name + "!";
}
```

JAVASCRIPT

```
// 기본값 파라미터 지원
function greet(name, message = "안녕") {
    return `${message}, ${name}!`;
}

greet("홍길동"); // "안녕, 홍길동!"
greet("홍길동", "반가워"); // "반가워, 홍길동!"
```

나머지 파라미터 (Rest Parameters)

JAVA

```
// 가변 인자
public int sum(int... numbers) {
    int total = 0;
    for (int n : numbers) {
        total += n;
    }
    return total;
}

sum(1, 2, 3); // 6
```

JAVASCRIPT

```
// 나머지 파라미터
function sum(...numbers) {
    return numbers.reduce((a, b) => a + b, 0);
}

sum(1, 2, 3); // 6

// 일부 파라미터 + 나머지
function log(level, ...messages) {
    console.log(`[${level}]`, ...messages);
}
```

arguments 객체 (레거시)

```
// ES6 이전에 사용하던 방식 (권장하지 않음)
function oldSum() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) {
    total += arguments[i];
  }
  return total;
}

// 화살표 함수에서는 arguments 사용 불가
const arrowSum = () => {
  // console.log(arguments); // ReferenceError
};

// 항상 나머지 파라미터 사용
const modernSum = (...args) => args.reduce((a, b) => a + b, 0);
```

콜백 함수

함수를 다른 함수의 인자로 전달하는 패턴입니다. JavaScript에서 매우 흔하게 사용됩니다.

JAVA

```
// 함수형 인터페이스로 콜백 구현
list.forEach(item -> {
  System.out.println(item);
});

// 또는 메서드 레퍼런스
list.forEach(System.out::println);

// Comparator로 정렬
list.sort((a, b) -> a.compareTo(b));
```

JAVASCRIPT

```
// 함수를 직접 전달
list.forEach(item => {
  console.log(item);
});

// 또는 함수 이름으로 전달
list.forEach(console.log);

// 정렬
list.sort((a, b) => a - b);
```

```
// 콜백 패턴 예제
function fetchData(callback) {
  // 데이터를 가져온 후 콜백 호출
  const data = { name: "홍길동" };
  callback(data);
}

fetchData(data => {
  console.log(data.name); // "홍길동"
});

// 에러 처리 콜백 (Node.js 스타일)
function readFile(path, callback) {
  // callback(error, data) 형태
  if (error) {
    callback(error, null);
  } else {
    callback(null, fileData);
  }
}

readFile("test.txt", (err, data) => {
  if (err) {
    console.error("에러:", err);
    return;
  }
  console.log(data);
});
```

클로저 (Closure)

클로저는 함수가 선언될 때의 렉시컬 환경을 기억하는 것입니다. Java에서는 비슷한 개념이 없어 처음에는 어려울 수 있습니다.

```
function createCounter() {
  let count = 0; // 외부에서 접근 불가

  return {
    increment() {
      count++;
      return count;
    },
    decrement() {
      count--;
      return count;
    },
    getCount() {
      return count;
    }
  };
}

const counter = createCounter();
console.log(counter.getCount()); // 0
console.log(counter.increment()); // 1
console.log(counter.increment()); // 2
console.log(counter.decrement()); // 1

// count 변수에 직접 접근 불가능
// console.log(counter.count); // undefined
```

클로저의 핵심

내부 함수가 외부 함수의 변수에 접근할 수 있으며, 외부 함수가 종료된 후에도 그 변수들이 살아 있습니다. 이를 통해 private 변수를 구현할 수 있습니다.

JAVA

```
// private 변수는 클래스로 구현
public class Counter {
    private int count = 0;

    public int increment() {
        return ++count;
    }

    public int getCount() {
        return count;
    }
}
```

JAVASCRIPT

```
// 클로저로 private 변수 구현
function createCounter() {
    let count = 0; // private

    return {
        increment: () => ++count,
        getCount: () => count
    };
}
```

클로저 활용 예제

```

// 1. 함수 팩토리
function createMultiplier(multiplier) {
  return function(number) {
    return number * multiplier;
  };
}

const double = createMultiplier(2);
const triple = createMultiplier(3);

console.log(double(5)); // 10
console.log(triple(5)); // 15

// 2. 메모이제이션 (캐싱)
function memoize(fn) {
  const cache = {};

  return function(...args) {
    const key = JSON.stringify(args);
    if (cache[key] === undefined) {
      cache[key] = fn(...args);
    }
    return cache[key];
  };
}

const expensiveCalculation = memoize(n => {
  console.log("계산 중...");
  return n * n;
});

expensiveCalculation(5); // "계산 중..." → 25
expensiveCalculation(5); // 캐시에서 반환 → 25

```

클로저 주의점: 루프에서의 var

```
// var를 사용한 잘못된 예
for (var i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log(i); // 3, 3, 3 출력!
  }, 100);
}
// var는 함수 스코프이므로 i는 하나만 존재

// let을 사용한 올바른 예
for (let i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log(i); // 0, 1, 2 출력
  }, 100);
}
// let은 블록 스코프이므로 각 반복마다 새로운 i
```

this 바인딩

Java에서 **this** 는 항상 현재 인스턴스를 가리킵니다. JavaScript에서 **this** 는 함수가 어떻게 호출되었는지에 따라 달라집니다.

```
const user = {
  name: "홍길동",

  // 일반 함수: this는 호출 객체
  greet() {
    console.log(`안녕, ${this.name}!`);
  },

  // 화살표 함수: this는 상위 스코프
  greetArrow: () => {
    console.log(`안녕, ${this.name}!`); // this는 user가 아님!
  }
};

user.greet(); // "안녕, 홍길동!"
user.greetArrow(); // "안녕, undefined!" (또는 전역 객체의 name)
```

this가 달라지는 경우

```
const user = {
  name: "홍길동",
  greet() {
    console.log(`안녕, ${this.name}!`);
  }
};

// 1. 메서드로 호출 - this는 user
user.greet(); // "안녕, 홍길동!"

// 2. 함수로 추출하여 호출 - this가 사라짐
const fn = user.greet;
fn(); // "안녕, undefined!" (strict mode에서는 에러)

// 3. 콜백으로 전달 - this가 사라짐
setTimeout(user.greet, 100); // "안녕, undefined!"
```

this 바인딩 해결 방법

```

const user = {
  name: "홍길동",
  greet() {
    console.log(`안녕, ${this.name}!`);
  }
};

// 방법 1: bind() 사용
const boundGreet = user.greet.bind(user);
setTimeout(boundGreet, 100); // "안녕, 홍길동!"

// 방법 2: 화살표 함수로 감싸기
setTimeout(() => user.greet(), 100); // "안녕, 홍길동!"

// 방법 3: 클래스에서 화살표 함수 사용
class User {
  name = "홍길동";

  // 화살표 함수로 메서드 정의
  greet = () => {
    console.log(`안녕, ${this.name}!`);
  };
}

const u = new User();
setTimeout(u.greet, 100); // "안녕, 홍길동!"

```

고차 함수 (Higher-Order Function)

함수를 인자로 받거나 함수를 반환하는 함수입니다. Java 8의 함수형 프로그래밍과 유사합니다.

```

// 함수를 인자로 받는 고차 함수
function repeat(n, action) {
  for (let i = 0; i < n; i++) {
    action(i);
  }
}

repeat(3, console.log); // 0, 1, 2

// 함수를 반환하는 고차 함수
function greaterThan(n) {
  return m => m > n;
}

const greaterThan10 = greaterThan(10);
console.log(greaterThan10(11)); // true
console.log(greaterThan10(9)); // false

// 배열 메서드와 함께 사용
const numbers = [1, 2, 3, 4, 5];

// map: 변환
const doubled = numbers.map(x => x * 2); // [2, 4, 6, 8, 10]

// filter: 필터링
const evens = numbers.filter(x => x % 2 === 0); // [2, 4]

// reduce: 축약
const sum = numbers.reduce((acc, x) => acc + x, 0); // 15

// 체이닝
const result = numbers
  .filter(x => x % 2 === 1) // 홀수만
  .map(x => x * x) // 제곱
  .reduce((a, b) => a + b); // 합계
// 1 + 9 + 25 = 35

```

즉시 실행 함수 (IIFE)

함수를 정의하자마자 실행하는 패턴입니다. 스코프 격리에 사용됩니다.

```
// 즉시 실행 함수 표현식 (IIFE)
(function() {
    const privateVar = "비밀";
    console.log(privateVar);
})();

// 화살표 함수 버전
(() => {
    const privateVar = "비밀";
    console.log(privateVar);
})();

// 값을 반환받을 수도 있음
const result = (() => {
    const a = 10;
    const b = 20;
    return a + b;
})();

console.log(result); // 30
```

ES6 이전에는 변수 스코프를 만들기 위해 IIFE를 많이 사용했습니다. 지금은 `let` / `const` 의 블록 스코프로 대체되어 사용 빈도가 줄었습니다.

정리

JavaScript의 함수는 일급 객체 - 변수에 할당, 인자로 전달, 반환값으로 사용 가능

함수 선언문은 호이스팅됨, 함수 표현식과 화살표 함수는 호이스팅되지 않음

화살표 함수는 간결하지만 자체 `this` 가 없음 (상위 스코프의 `this` 사용)

기본값 파라미터, 나머지 파라미터로 유연한 함수 정의 가능

클로저: 함수가 선언 시점의 스코프를 기억함 (private 변수 구현에 활용)

`this` 는 호출 방식에 따라 달라짐 - `bind()` 나 화살표 함수로 고정

고차 함수로 함수형 프로그래밍 패턴 구현 (map, filter, reduce 등)

객체와 배열

JavaScript의 객체는 Java와 달리 클래스 없이 생성할 수 있습니다. 객체 리터럴, 프로토타입, 배열 메서드, 그리고 구조 분해 할당을 학습합니다.

객체 리터럴

JavaScript에서 객체를 만드는 가장 간단한 방법입니다. Java의 Map과 비슷하지만 훨씬 유연합니다.

JAVA

```
// 클래스 정의 필요
public class User {
    private String name;
    private int age;

    public User(String name, int age) {
        this.name = name;
        this.age = age;
    }
    // getters, setters...
}

User user = new User("홍길동", 30);
```

JAVASCRIPT

```
// 클래스 없이 객체 생성
const user = {
  name: "홍길동",
  age: 30
};

// 즉시 사용 가능
console.log(user.name); // "홍길동"
```

프로퍼티 접근

```
const user = {
  name: "홍길동",
  age: 30,
  "e-mail": "hong@example.com" // 특수문자가 있으면 따옴표 필요
};

// 점 표기법
console.log(user.name); // "홍길동"

// 대괄호 표기법
console.log(user["name"]); // "홍길동"
console.log(user["e-mail"]); // "hong@example.com"

// 동적 프로퍼티 접근
const key = "age";
console.log(user[key]); // 30

// 프로퍼티 추가/수정
user.address = "서울"; // 새 프로퍼티 추가
user.age = 31; // 기존 프로퍼티 수정

// 프로퍼티 삭제
delete user.address;
```

메서드 정의

```

const user = {
  name: "홍길동",

  // ES5 방식
  greet: function() {
    return `안녕, ${this.name}!`;
  },

  // ES6 축약 문법 (권장)
  sayHello() {
    return `Hello, ${this.name}!`;
  }
};

console.log(user.greet()); // "안녕, 홍길동!"
console.log(user.sayHello()); // "Hello, 홍길동!"

```

계산된 프로퍼티 이름

```

const prefix = "user";
const id = 1;

// ES6: 계산된 프로퍼티 이름
const obj = {
  [prefix + id]: "홍길동",
  [`${prefix}${id + 1}`]: "김철수"
};

console.log(obj.user1); // "홍길동"
console.log(obj.user2); // "김철수"

```

단축 프로퍼티

```

const name = "홍길동";
const age = 30;

// ES5
const user1 = {
  name: name,
  age: age
};

// ES6 단축 문법
const user2 = {
  name,    // name: name과 동일
  age     // age: age와 동일
};

// 함수에서 객체 반환할 때 유용
function createUser(name, age) {
  return { name, age };
}

const user3 = createUser("김철수", 25);
// { name: "김철수", age: 25 }

```

객체 복사와 비교

JAVA

```

// 참조 복사
User a = new User("홍길동");
User b = a;
// a와 b는 같은 객체

// 값 비교
a.equals(b); // true

// 깊은 복사
User c = a.clone();

```

JAVASCRIPT

```
// 참조 복사
const a = { name: "홍길동" };
const b = a;
// a와 b는 같은 객체

// 참조 비교
a === b; // true

// 얕은 복사
const c = { ...a };
const d = Object.assign({}, a);
```

```
// 객체 비교
const obj1 = { name: "홍길동" };
const obj2 = { name: "홍길동" };
const obj3 = obj1;

obj1 === obj2; // false (다른 객체)
obj1 === obj3; // true (같은 참조)

// 내용 비교하려면 직접 구현
JSON.stringify(obj1) === JSON.stringify(obj2); // true (주의: 순서 의존)

// 얕은 복사 (shallow copy)
const original = { name: "홍길동", address: { city: "서울" } };
const copied = { ...original };

copied.name = "김철수"; // original에 영향 없음
copied.address.city = "부산"; // original도 변경됨! (중첩 객체는 참조)

// 깊은 복사 (deep copy)
const deepCopied = JSON.parse(JSON.stringify(original));
// 또는 structuredClone(original) - 최신 브라우저
```

얕은 복사 주의

스프레드 연산자(`...`)나 `Object.assign()`은 1단계만 복사합니다. 중첩된 객체는 여전히 같은 참조를 공유합니다.

프로토타입

JavaScript는 클래스 기반이 아닌 프로토타입 기반 언어입니다. 모든 객체는 다른 객체를 프로토타입으로 가지며, 그 객체의 프로퍼티를 상속받습니다.

```
// 모든 객체는 Object.prototype을 상속
const obj = { name: "홍길동" };

// obj에는 toString이 없지만 사용 가능
obj.toString(); // "[object Object]"

// 프로토타입 체인 확인
Object.getPrototypeOf(obj) === Object.prototype; // true

// 배열의 프로토타입
const arr = [1, 2, 3];
Object.getPrototypeOf(arr) === Array.prototype; // true

// Array.prototype은 Object.prototype을 상속
Object.getPrototypeOf(Array.prototype) === Object.prototype; // true
```

프로토타입 체인

객체에서 프로퍼티를 찾을 때, 해당 객체에 없으면 프로토타입에서 찾습니다. 프로토타입에도 없으면 그 프로토타입의 프로토타입에서 찾습니다. 이 과정이 `null`에 도달할 때까지 계속됩니다.

```
// 프로토타입으로 상속 구현 (ES6 이전 방식)
function Animal(name) {
  this.name = name;
}

Animal.prototype.speak = function() {
  console.log(`${this.name} makes a sound.`);
};

function Dog(name, breed) {
  Animal.call(this, name); // 부모 생성자 호출
  this.breed = breed;
}

// 프로토타입 상속
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;

Dog.prototype.bark = function() {
  console.log(`${this.name} barks!`);
};

const dog = new Dog("멍멍이", "진돗개");
dog.speak(); // "멍멍이 makes a sound."
dog.bark();  // "멍멍이 barks!"
```

위 코드는 복잡합니다. ES6 클래스 문법으로 같은 것을 더 쉽게 구현할 수 있습니다(5장에서 다룹니다).

배열

JavaScript의 배열은 Java의 ArrayList와 비슷하지만 더 유연합니다.

JAVA

```
// 배열
int[] arr = {1, 2, 3};
int[] arr2 = new int[3];

// ArrayList
List<Integer> list = new ArrayList<>();
list.add(1);
list.add(2);
list.get(0); // 1
list.size(); // 2
```

JAVASCRIPT

```
// 배열 리터럴
const arr = [1, 2, 3];
const arr2 = new Array(3);

// 동적으로 조작
arr.push(4); // 끝에 추가
arr[0]; // 1
arr.length; // 4

// 타입 혼합 가능
const mixed = [1, "two", true];
```

배열 기본 조작

```

const arr = [1, 2, 3];

// 추가
arr.push(4);          // 끝에 추가 → [1, 2, 3, 4]
arr.unshift(0);       // 앞에 추가 → [0, 1, 2, 3, 4]

// 제거
arr.pop();            // 끝에서 제거 → [0, 1, 2, 3]
arr.shift();          // 앞에서 제거 → [1, 2, 3]

// 특정 위치 조작
arr.splice(1, 1);      // 인덱스 1에서 1개 제거 → [1, 3]
arr.splice(1, 0, 2);   // 인덱스 1에 2 삽입 → [1, 2, 3]
arr.splice(1, 1, "two"); // 인덱스 1의 값을 교체 → [1, "two", 3]

// 검색
const idx = arr.indexOf(2); // 인덱스 반환, 없으면 -1
const found = arr.includes(2); // true/false
const item = arr.find(x => x > 1); // 조건에 맞는 첫 요소
const items = arr.filter(x => x > 1); // 조건에 맞는 모든 요소

// 정렬
arr.sort();            // 문자열로 정렬 (주의!)
arr.sort((a, b) => a - b); // 숫자 오름차순
arr.reverse();         // 뒤집기

```

sort() 주의

`sort()` 는 기본적으로 문자열로 변환 후 정렬합니다. `[10, 2, 1].sort()` 는 `[1, 10, 2]` 가 됩니다! 숫자 정렬은 반드시 비교 함수를 전달하세요.

배열 고차 함수

배열 메서드 중 함수를 인자로 받는 메서드들입니다. Java Stream API와 유사합니다.

map - 변환

JAVA

```
List<Integer> numbers = List.of(1, 2, 3);

List<Integer> doubled = numbers.stream()
    .map(x -> x * 2)
    .collect(Collectors.toList());
// [2, 4, 6]
```

JAVASCRIPT

```
const numbers = [1, 2, 3];

const doubled = numbers.map(x => x * 2);
// [2, 4, 6]

// 원본 배열은 변경되지 않음
console.log(numbers); // [1, 2, 3]
```

filter - 필터링

JAVA

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5);

List<Integer> evens = numbers.stream()
    .filter(x -> x % 2 == 0)
    .collect(Collectors.toList());
// [2, 4]
```

JAVASCRIPT

```
const numbers = [1, 2, 3, 4, 5];

const evens = numbers.filter(x => x % 2 === 0);
// [2, 4]
```

reduce - 축약

JAVA

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5);

int sum = numbers.stream()
    .reduce(0, (a, b) -> a + b);
// 15
```

JAVASCRIPT

```
const numbers = [1, 2, 3, 4, 5];

const sum = numbers.reduce((acc, cur) => acc + cur, 0);
// 15

// 객체로 변환
const users = [{name: "홍", age: 30}, {name: "김", age: 25}];
const byName = users.reduce((acc, user) => {
    acc[user.name] = user;
    return acc;
}, {});
// { "홍": {...}, "김": {...} }
```

기타 유용한 메서드

```

const numbers = [1, 2, 3, 4, 5];

// forEach - 순회 (반환값 없음)
numbers.forEach(n => console.log(n));

// some - 하나라도 만족하면 true
numbers.some(n => n > 3); // true

// every - 모두 만족해야 true
numbers.every(n => n > 0); // true

// find - 조건에 맞는 첫 요소
numbers.find(n => n > 3); // 4

// findIndex - 조건에 맞는 첫 인덱스
numbers.findIndex(n => n > 3); // 3

// flat - 중첩 배열 평탄화
[[1, 2], [3, 4]].flat(); // [1, 2, 3, 4]

// flatMap - map 후 flat
[1, 2].flatMap(x => [x, x * 2]); // [1, 2, 2, 4]

```

메서드 체이닝

```

const users = [
  { name: "홍길동", age: 30, active: true },
  { name: "김철수", age: 25, active: false },
  { name: "이영희", age: 35, active: true },
  { name: "박민수", age: 28, active: true }
];

// 활성 사용자 중 30세 이상의 이름 목록
const result = users
  .filter(u => u.active) // 활성 사용자만
  .filter(u => u.age >= 30) // 30세 이상
  .map(u => u.name) // 이름만 추출
  .sort(); // 정렬

console.log(result); // ["이영희", "홍길동"]

```

구조 분해 할당 (Destructuring)

객체나 배열에서 값을 추출하여 변수에 할당하는 문법입니다. Java에는 없는 강력한 기능입니다.

객체 구조 분해

```
const user = {
  name: "홍길동",
  age: 30,
  address: {
    city: "서울",
    zip: "12345"
  }
};

// 기본 구조 분해
const { name, age } = user;
console.log(name); // "홍길동"
console.log(age);  // 30

// 다른 변수명으로 할당
const { name: userName } = user;
console.log(userName); // "홍길동"

// 기본값 설정
const { email = "없음" } = user;
console.log(email); // "없음"

// 중첩 구조 분해
const { address: { city } } = user;
console.log(city); // "서울"

// 나머지 프로퍼티
const { name: n, ...rest } = user;
console.log(rest); // { age: 30, address: {...} }
```

배열 구조 분해

```

const arr = [1, 2, 3, 4, 5];

// 기본 구조 분해
const [first, second] = arr;
console.log(first); // 1
console.log(second); // 2

// 요소 건너뛰기
const [, , third] = arr;
console.log(third); // 3

// 나머지 요소
const [head, ...tail] = arr;
console.log(head); // 1
console.log(tail); // [2, 3, 4, 5]

// 기본값
const [a, b, c, d, e, f = 0] = arr;
console.log(f); // 0

// 변수 교환
let x = 1, y = 2;
[x, y] = [y, x];
console.log(x, y); // 2, 1

```

함수 파라미터에서 구조 분해

JAVA

```

// Java는 객체를 받아서 각각 접근
public void greet(User user) {
    String name = user.getName();
    int age = user.getAge();
    System.out.println(name + ", " + age);
}

```

JAVASCRIPT

```
// 파라미터에서 바로 구조 분해
function greet({ name, age }) {
  console.log(`${name}, ${age}`);
}

greet({ name: "홍길동", age: 30 });

// 기본값과 함께
function config({ port = 3000, host = "localhost" } = {}) {
  console.log(`${host}:${port}`);
}

config(); // localhost:3000
config({ port: 8080 }); // localhost:8080
```

스프레드 연산자 (...)

배열이나 객체를 펼치는 연산자입니다. Java에는 없는 문법입니다.

배열 스프레드

```
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];

// 배열 합치기
const combined = [...arr1, ...arr2]; // [1, 2, 3, 4, 5, 6]

// Java의 경우
// List.of(arr1, arr2).stream().flatMap(...).collect(...)

// 배열 복사
const copy = [...arr1]; // [1, 2, 3]

// 중간에 삽입
const inserted = [...arr1.slice(0, 1), 99, ...arr1.slice(1)];
// [1, 99, 2, 3]

// 함수 인자로 전달
function sum(a, b, c) {
    return a + b + c;
}
sum(...arr1); // 6

// Math.max와 함께
Math.max(...arr1); // 3
```

객체 스프레드

```

const defaults = {
  theme: "light",
  fontSize: 14,
  language: "ko"
};

const userSettings = {
  theme: "dark",
  fontSize: 16
};

// 객체 병합 (뒤에 오는 값이 우선)
const settings = { ...defaults, ...userSettings };
// { theme: "dark", fontSize: 16, language: "ko" }

// 특정 프로퍼티 덮어쓰기
const updated = { ...settings, theme: "blue" };

// 프로퍼티 제거
const { theme, ...withoutTheme } = settings;
console.log(withoutTheme); // { fontSize: 16, language: "ko" }

// 조건부 프로퍼티 추가
const user = {
  name: "홍길동",
  ...(isAdmin && { role: "admin" })
};

```

유용한 객체 메서드

```

const user = { name: "홍길동", age: 30 };

// 키 배열
Object.keys(user);    // ["name", "age"]

// 값 배열
Object.values(user);  // ["홍길동", 30]

// [키, 값] 쌍 배열
Object.entries(user); // [["name", "홍길동"], ["age", 30]]

// 배열에서 객체 생성
Object.fromEntries([["name", "홍길동"], ["age", 30]]);
// { name: "홍길동", age: 30 }

// 객체 순회
for (const [key, value] of Object.entries(user)) {
  console.log(`${key}: ${value}`);
}

// 프로퍼티 존재 확인
"name" in user;          // true
user.hasOwnProperty("name"); // true

// 객체 병합
Object.assign({}, user, { email: "hong@test.com" });

// 객체 동결 (변경 불가)
const frozen = Object.freeze({ x: 1 });
frozen.x = 2; // 무시됨 (strict mode에서는 에러)

// 객체 봉인 (프로퍼티 추가/삭제 불가, 수정은 가능)
const sealed = Object.seal({ x: 1 });
sealed.x = 2; // OK
sealed.y = 3; // 무시됨

```

정리

객체 리터럴 `{}` 로 클래스 없이 객체 생성 가능

프로퍼티 접근: 점 표기법 (`obj.key`) 또는 대괄호 (`obj["key"]`)

ES6 단축 문법: `{ name, age }`, 메서드 `greet() {}`

모든 객체는 프로토타입 체인을 통해 상속받음

배열 고차 함수: `map`, `filter`, `reduce`, `find` 등

구조 분해: `const { name } = obj , const [a, b] = arr`

스프레드: `[...arr1, ...arr2] , {...obj1, ...obj2}`

얕은 복사 주의: 중첩 객체는 참조가 공유됨

클래스와 모듈

ES6에서 도입된 클래스 문법은 Java 개발자에게 친숙합니다. 클래스 정의, 상속, 그리고 모듈 시스템을 학습합니다.

ES6 클래스

ES6 이전에는 함수와 프로토타입으로 "클래스"를 흉내냈습니다. ES6부터는 Java와 비슷한 `class` 키워드를 사용할 수 있습니다.

JAVA

```
public class User {  
    private String name;  
    private int age;  
  
    public User(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String greet() {  
        return "안녕, " + this.name + "!";  
    }  
}
```

JAVASCRIPT

```

class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    return `안녕, ${this.name}!`;
  }
}

const user = new User("홍길동", 30);
user.greet(); // "안녕, 홍길동!"

```

클래스는 문법적 설탕

JavaScript의 클래스는 내부적으로 여전히 프로토타입을 사용합니다. Java의 클래스와 비슷해 보이지만, 동작 방식은 다릅니다. 특히 `this` 바인딩에 주의해야 합니다.

클래스 표현식

```

// 클래스 선언문
class User { }

// 클래스 표현식 (이름 있음)
const User = class UserClass { };

// 클래스 표현식 (익명)
const User = class { };

// 함수처럼 변수에 할당 가능
const createClass = () => class {
  constructor(name) {
    this.name = name;
  }
};

```

생성자와 프로퍼티

constructor

```
class User {  
    // 생성자 - 클래스당 하나만 가능  
    constructor(name, age) {  
        this.name = name;  
        this.age = age;  
    }  
}  
  
// Java처럼 오버로딩 불가  
// class User {  
//     constructor(name) { }  
//     constructor(name, age) { } // SyntaxError  
// }  
  
// 대신 기본값으로 처리  
class User {  
    constructor(name, age = 0) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

인스턴스 프로퍼티

JAVA

```
public class User {  
    // 필드 선언  
    private String name;  
    private int age = 0;  
  
    public User(String name) {  
        this.name = name;  
    }  
}
```

JAVASCRIPT

```

class User {
  // 클래스 필드 (ES2022)
  name;
  age = 0;

  constructor(name) {
    this.name = name;
  }
}

// 또는 constructor에서만 정의
class User {
  constructor(name) {
    this.name = name;
    this.age = 0;
  }
}

```

Private 필드 (#)

ES2022부터 **#** 접두사로 진짜 private 필드를 만들 수 있습니다.

JAVA

```

public class BankAccount {
  private double balance = 0;

  public void deposit(double amount) {
    this.balance += amount;
  }

  public double getBalance() {
    return this.balance;
  }
}

```

JAVASCRIPT

```

class BankAccount {
  #balance = 0; // private 필드

  deposit(amount) {
    this.#balance += amount;
  }

  getBalance() {
    return this.#balance;
  }
}

const account = new BankAccount();
account.deposit(1000);
account.getBalance(); // 1000
// account.#balance; // SyntaxError

```

```

// 관례적 private (실제로는 public)
class User {
  _password; // 밑줄 접두사는 관례일 뿐, 접근 가능
}

// 진짜 private
class User {
  #password; // 외부에서 절대 접근 불가

  #validatePassword(input) { // private 메서드
    return input === this.#password;
  }
}

```

메서드

인스턴스 메서드

```

class Calculator {
  constructor(value = 0) {
    this.value = value;
  }

  add(n) {
    this.value += n;
    return this; // 메서드 체이닝을 위해 this 반환
  }

  subtract(n) {
    this.value -= n;
    return this;
  }

  getResult() {
    return this.value;
  }
}

const calc = new Calculator(10);
const result = calc.add(5).subtract(3).getResult(); // 12

```

정적 메서드 (static)

JAVA

```

public class MathUtils {
  public static int max(int a, int b) {
    return a > b ? a : b;
  }
}

MathUtils.max(10, 20); // 20

```

JAVASCRIPT

```

class MathUtils {
    static max(a, b) {
        return a > b ? a : b;
    }

    static PI = 3.14159; // 정적 프로퍼티
}

MathUtils.max(10, 20); // 20
MathUtils.PI;          // 3.14159

```

```

// 정적 메서드 활용 예
class User {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }

    // 팩토리 메서드
    static fromJSON(json) {
        const data = JSON.parse(json);
        return new User(data.name, data.age);
    }

    // 유틸리티 메서드
    static isAdult(user) {
        return user.age >= 18;
    }
}

const user = User.fromJSON('{"name":"홍길동","age":30}');
User.isAdult(user); // true

```

Getter와 Setter

JAVA

```

public class Circle {
    private double radius;

    public double getRadius() {
        return radius;
    }

    public void setRadius(double r) {
        if (r < 0) throw new IllegalArgumentException();
        this.radius = r;
    }

    public double getArea() {
        return Math.PI * radius * radius;
    }
}

```

JAVASCRIPT

```

class Circle {
    #radius = 0;

    get radius() {
        return this.#radius;
    }

    set radius(value) {
        if (value < 0) throw new Error("음수 불가");
        this.#radius = value;
    }

    get area() { // 읽기 전용
        return Math.PI * this.#radius ** 2;
    }
}

const c = new Circle();
c.radius = 5; // setter 호출
c.radius;    // getter 호출 → 5
c.area;     // getter 호출 → 78.54...

```

상속

JAVA

```
public class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public void speak() {
        System.out.println(name + " makes a sound");
    }
}

public class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }

    @Override
    public void speak() {
        System.out.println(name + " barks");
    }
}
```

JAVASCRIPT

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound`);
  }
}

class Dog extends Animal {
  constructor(name) {
    super(name); // 반드시 첫 줄에서 호출
  }

  speak() { // 메서드 오버라이딩
    console.log(`${this.name} barks`);
  }
}

const dog = new Dog("멍멍이");
dog.speak(); // "멍멍이 barks"
```

super 키워드

```

class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    return `${this.name} makes a sound`;
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    // 자식 클래스에서 this 사용 전에 super() 필수
    super(name);
    this.breed = breed;
  }

  speak() {
    // 부모 메서드 호출
    const parentResult = super.speak();
    return `${parentResult}. Actually, ${this.name} barks!`;
  }

  getInfo() {
    return `${this.name} is a ${this.breed}`;
  }
}

const dog = new Dog("멍멍이", "진돗개");
dog.speak(); // "멍멍이 makes a sound. Actually, 멍멍이 barks!"
dog.getInfo(); // "멍멍이 is a 진돗개"

```

super() 호출 규칙

자식 클래스의 constructor에서 **this**를 사용하기 전에 반드시 **super()**를 호출해야 합니다. 호출하지 않으면 ReferenceError가 발생합니다.

instanceof 연산자

```
class Animal { }
class Dog extends Animal { }
class Cat extends Animal { }

const dog = new Dog();

dog instanceof Dog;    // true
dog instanceof Animal; // true
dog instanceof Cat;    // false
dog instanceof Object; // true (모든 객체는 Object의 인스턴스)
```

Java 클래스와의 차이점

기능	Java	JavaScript
인터페이스 interface 키워드		없음 (TypeScript에서 지원)
추상 클래스 abstract 키워드		없음 (직접 구현)
접근 제어자 public, private, protected	# (private만, ES2022)	
다중 상속	인터페이스로 가능	없음 (Mixin 패턴 사용)
오버로딩	지원	없음 (기본값/rest 파라미터)
final 클래스 final 키워드		없음

추상 클래스 흉내내기

```

class Shape {
  constructor() {
    if (new.target === Shape) {
      throw new Error("Shape은 직접 인스턴스화할 수 없습니다");
    }
  }

  // 추상 메서드 (자식에서 구현 필수)
  getArea() {
    throw new Error("getArea()를 구현해야 합니다");
  }
}

class Circle extends Shape {
  constructor(radius) {
    super();
    this.radius = radius;
  }

  getArea() {
    return Math.PI * this.radius ** 2;
  }
}

// new Shape(); // Error: Shape은 직접 인스턴스화할 수 없습니다
new Circle(5).getArea(); // 78.54...

```

모듈 시스템

ES6 이전에는 JavaScript에 공식 모듈 시스템이 없었습니다. CommonJS(Node.js), AMD 등 여러 방식이 사용되었습니다. ES6부터 표준 모듈 시스템이 도입되었습니다.

export - 내보내기

```

// math.js

// 개별 내보내기 (Named Export)
export const PI = 3.14159;

export function add(a, b) {
    return a + b;
}

export class Calculator {
    // ...
}

// 한 번에 내보내기
const subtract = (a, b) => a - b;
const multiply = (a, b) => a * b;

export { subtract, multiply };

// 이름 바꿔서 내보내기
export { subtract as minus };

```

export default - 기본 내보내기

```

// User.js

// 기본 내보내기 (모듈당 하나만)
export default class User {
    constructor(name) {
        this.name = name;
    }
}

// 또는
class User { }
export default User;

// 함수도 가능
export default function(name) {
    return { name };
}

```

import - 가져오기

JAVA

```
// 개별 import
import java.util.List;
import java.util.ArrayList;

// 패키지 전체
import java.util.*;

// 정적 import
import static java.lang.Math.PI;
```

JAVASCRIPT

```
// Named import
import { add, subtract } from './math.js';

// 이름 바꿔서 import
import { add as plus } from './math.js';

// 전체 import
import * as math from './math.js';
math.add(1, 2);

// Default import
import User from './User.js';

// 혼합
import User, { add } from './module.js';
```

```

// 다양한 import 패턴

// Named exports 가져오기
import { PI, add, Calculator } from './math.js';

// 이름 변경
import { add as sum } from './math.js';

// 모든 Named exports를 네임스페이스로
import * as MathUtils from './math.js';
MathUtils.PI;
MathUtils.add(1, 2);

// Default export 가져오기
import User from './User.js';

// Default와 Named 함께
import User, { add, subtract } from './module.js';

// 부작용만 실행 (값을 가져오지 않음)
import './init.js';

```

re-export

```

// index.js - 여러 모듈을 하나로 모아서 re-export

export { add, subtract } from './math.js';
export { default as User } from './User.js';
export * from './utils.js';

// 사용하는 쪽
import { add, User } from './index.js';

```

동적 import

```
// 조건부 로딩
if (condition) {
  const module = await import('./heavy-module.js');
  module.doSomething();
}

// 버튼 클릭 시 로딩
button.addEventListener('click', async () => {
  const { showModal } = await import('./modal.js');
  showModal();
});

// 경로 동적 생성
const lang = 'ko';
const messages = await import(`./locales/${lang}.js`);
```

모듈 사용하기

브라우저에서

```
<!-- type="module" 필수 -->
<script type="module">
  import { add } from './math.js';
  console.log(add(1, 2));
</script>

<!-- 외부 파일 -->
<script type="module" src="./main.js"></script>

<!-- 모듈 미지원 브라우저용 폴백 -->
<script nomodule src="./fallback.js"></script>
```

Node.js에서

```
// package.json에서 타입 지정
{
  "type": "module"
}

// 또는 파일 확장자를 .mjs 사용
// math.mjs, main.mjs

// ES 모듈 사용
import { add } from './math.js';

// CommonJS와 혼용
import pkg from './commonjs-module.cjs';

// Node.js 내장 모듈
import fs from 'fs';
import { readFile } from 'fs/promises';
```

CommonJS vs ES Modules

특성	CommonJS	ES Modules
문법	require(), module.exports	import, export
로딩	동기 (런타임)	비동기 (정적 분석)
환경	Node.js 기본	브라우저, Node.js
트리 셰이킹	어려움	지원
Top-level await	불가	가능

```
// CommonJS (레거시, Node.js)
const math = require('./math');
module.exports = { add, subtract };

// ES Modules (현대적, 권장)
import { add } from './math.js';
export { add, subtract };
```

ES Modules 권장

새로운 프로젝트에서는 ES Modules를 사용하세요. 정적 분석이 가능해서 번들러가 사용하지 않는 코드를 제거(tree-shaking)할 수 있습니다. 브라우저에서도 네이티브로 지원됩니다.

실전 패턴

모듈 구조 예시

```
src/
├─ index.js          # 진입점
├─ models/
│  ├─ index.js      # re-export
│  ├─ User.js
│  └─ Product.js
├─ services/
│  ├─ index.js
│  ├─ api.js
│  └─ auth.js
└─ utils/
   ├─ index.js
   └─ helpers.js

// models/index.js
export { default as User } from './User.js';
export { default as Product } from './Product.js';

// 사용
import { User, Product } from './models/index.js';
// 또는
import { User, Product } from './models'; // 번들러가 index.js 자동 찾음
```

싱글톤 패턴

```
// config.js
class Config {
  constructor() {
    this.apiUrl = 'https://api.example.com';
    this.timeout = 5000;
  }
}

// ES 모듈은 기본적으로 싱글톤
// 같은 모듈을 여러 번 import해도 같은 인스턴스
export default new Config();

// 사용
import config from './config.js';
console.log(config.apiUrl);
```

정리

ES6 클래스: `class`, `constructor`, `extends`, `super`

Private 필드: `#` 접두사 (ES2022)

정적 멤버: `static` 키워드

Getter/Setter: `get`, `set` 키워드

클래스는 프로토타입의 문법적 설탕

Named export: `export { name }`

Default export: `export default` (모듈당 하나)

Import: `import { name } from './module.js'`

동적 import: `await import('./module.js')`

ES Modules를 기본으로 사용 (CommonJS는 레거시)

비동기 프로그래밍

JavaScript는 싱글스레드지만 비동기 처리로 효율적입니다. 콜백, Promise, async/await를 이해하고 Java의 CompletableFuture와 비교합니다.

동기 vs 비동기

Java에서는 대부분의 작업이 동기적으로 실행됩니다. 파일을 읽거나 네트워크 요청을 보내면 결과가 올 때까지 스레드가 대기합니다.

JavaScript는 싱글스레드이므로 대기하면 전체 프로그램이 멈춥니다. 대신 비동기 처리로 작업을 시작하고, 완료되면 콜백을 실행합니다.

JAVA (동기)

```
// 파일 읽기 - 완료될 때까지 대기
String content = Files.readString(path);
System.out.println(content);
System.out.println("다음 작업");

// 출력 순서: content → "다음 작업"
```

JAVASCRIPT (비동기)

```
// 파일 읽기 - 대기하지 않음
fs.readFile(path, (err, data) => {
  console.log(data);
});
console.log("다음 작업");

// 출력 순서: "다음 작업" → data
```

이벤트 루프

JavaScript는 이벤트 루프를 통해 비동기 작업을 처리합니다. 메인 스레드는 계속 실행되고, 비동기 작업이 완료되면 콜백이 큐에 추가되어 실행됩니다.

```
console.log("1");

setTimeout(() => {
  console.log("2");
}, 0); // 0ms 후 실행

console.log("3");

// 출력: 1 → 3 → 2
// setTimeout 콜백은 현재 실행 스택이 비워진 후에 실행됨
```

콜백 (Callback)

비동기 작업이 완료되면 실행할 함수를 전달하는 가장 기본적인 패턴입니다.

```
// 간단한 콜백 예제
function fetchUser(id, callback) {
  // 비동기 작업 시뮬레이션
  setTimeout(() => {
    const user = { id, name: "홍길동" };
    callback(user);
  }, 1000);
}

fetchUser(1, user => {
  console.log(user.name); // 1초 후 "홍길동"
});

// Node.js 스타일 (에러 우선 콜백)
function readFile(path, callback) {
  setTimeout(() => {
    if (path === "invalid") {
      callback(new Error("파일 없음"), null);
    } else {
      callback(null, "파일 내용");
    }
  }, 100);
}

readFile("test.txt", (err, data) => {
  if (err) {
    console.error("에러:", err.message);
    return;
  }
  console.log(data);
});
```

콜백 지옥 (Callback Hell)

연속된 비동기 작업은 콜백이 중첩되어 읽기 어려워집니다.

```
// 사용자 → 주문 → 상품 순서로 조회
getUser(userId, (err, user) => {
  if (err) {
    console.error(err);
    return;
  }
  getOrders(user.id, (err, orders) => {
    if (err) {
      console.error(err);
      return;
    }
    getProduct(orders[0].productId, (err, product) => {
      if (err) {
        console.error(err);
        return;
      }
      console.log(product);
      // 더 깊어질 수 있음...
    });
  });
});
```

콜백 지옥의 문제

1. 코드가 오른쪽으로 계속 들여쓰기됨
 2. 에러 처리가 각 콜백에서 반복됨
 3. 순서 이해와 디버깅이 어려움
- 이 문제를 해결하기 위해 Promise가 등장했습니다.

Promise

Promise는 비동기 작업의 최종 완료 또는 실패를 나타내는 객체입니다. ES6에서 도입되었습니다.

Promise 생성

```
// Promise 생성
const promise = new Promise((resolve, reject) => {
  // 비동기 작업 수행
  setTimeout(() => {
    const success = true;

    if (success) {
      resolve("성공!"); // 완료
    } else {
      reject(new Error("실패!")); // 실패
    }
  }, 1000);
});

// Promise 사용
promise
  .then(result => {
    console.log(result); // "성공!"
  })
  .catch(error => {
    console.error(error);
  });
```

Promise 상태

상태	설명
pending	대기 중 (초기 상태)
fulfilled	성공 (resolve 호출됨)
rejected	실패 (reject 호출됨)

then, catch, finally

```

fetchData()
  .then(data => {
    // 성공 시 실행
    console.log(data);
    return processData(data); // 다음 Promise 반환
  })
  .then(result => {
    // 이전 then의 반환값을 받음
    console.log(result);
  })
  .catch(error => {
    // 어디서든 에러 발생 시 실행
    console.error("에러:", error);
  })
  .finally(() => {
    // 성공/실패 관계없이 항상 실행
    console.log("완료");
  });

```

Promise 체이닝

```

// 콜백 지옥을 Promise로 개선
getUser(userId)
  .then(user => getOrders(user.id))
  .then(orders => getProduct(orders[0].productId))
  .then(product => {
    console.log(product);
  })
  .catch(error => {
    console.error(error); // 어디서든 에러 처리
  });

```

Promise 정적 메서드

```

// Promise.all - 모두 완료될 때까지 대기
const promises = [
  fetch('/api/users'),
  fetch('/api/products'),
  fetch('/api/orders')
];

Promise.all(promises)
  .then([users, products, orders]) => {
    // 모두 성공
  })
  .catch(error => {
    // 하나라도 실패하면 즉시 실행
  });

// Promise.allSettled - 모두 완료 (성공/실패 모두)
Promise.allSettled(promises)
  .then(results => {
    results.forEach(result => {
      if (result.status === 'fulfilled') {
        console.log(result.value);
      } else {
        console.log(result.reason);
      }
    });
  });

// Promise.race - 가장 빠른 것 하나만
Promise.race([
  fetch('/api/fast'),
  fetch('/api/slow')
]).then(first => {
  // 가장 먼저 완료된 것
});

// Promise.any - 가장 빠른 성공 하나
Promise.any(promises).then(first => {
  // 가장 먼저 성공한 것 (모두 실패하면 reject)
});

// Promise.resolve / Promise.reject
Promise.resolve("즉시 성공");
Promise.reject(new Error("즉시 실패"));

```

async/await

ES2017에서 도입된 문법으로, Promise를 동기 코드처럼 작성할 수 있습니다. 가장 권장되는 비동기 처리 방식입니다.

JAVA

```
// CompletableFuture
CompletableFuture<User> future =
    getUser(userId);

User user = future.get(); // 블로킹

// 또는 비동기 체이닝
getUser(userId)
    .thenApply(user -> getOrders(user.id))
    .thenApply(orders -> ...)
    .exceptionally(ex -> ...);
```

JAVASCRIPT

```
// async/await
async function main() {
    const user = await getUser(userId);
    const orders = await getOrders(user.id);
    const product = await getProduct(orders[0].id);
    console.log(product);
}

// 동기 코드처럼 읽히지만 실제로는 비동기
```

기본 문법

```

// async 함수는 항상 Promise를 반환
async function fetchUser(id) {
    const response = await fetch(`/api/users/${id}`);
    const user = await response.json();
    return user; // Promise로 감싸져서 반환
}

// 사용
fetchUser(1).then(user => console.log(user));

// 또는 다른 async 함수에서
async function main() {
    const user = await fetchUser(1);
    console.log(user);
}

// 화살표 함수
const fetchUser = async (id) => {
    const response = await fetch(`/api/users/${id}`);
    return response.json();
};

```

에러 처리

```

// try/catch 사용
async function fetchUser(id) {
  try {
    const response = await fetch(`/api/users/${id}`);

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const user = await response.json();
    return user;
  } catch (error) {
    console.error("사용자 조회 실패:", error);
    throw error; // 다시 던지거나 기본값 반환
  }
}

// 또는 호출하는 쪽에서 처리
async function main() {
  try {
    const user = await fetchUser(1);
    const orders = await getOrders(user.id);
  } catch (error) {
    console.error("에러 발생:", error);
  } finally {
    console.log("완료");
  }
}

```

병렬 실행

```

// 순차 실행 (느림)
async function sequential() {
  const user = await fetchUser(1);    // 1초
  const products = await fetchProducts(); // 1초
  // 총 2초
}

// 병렬 실행 (빠름)
async function parallel() {
  const [user, products] = await Promise.all([
    fetchUser(1),    // 동시에 시작
    fetchProducts()  // 동시에 시작
  ]);
  // 총 1초 (가장 느린 것 기준)
}

// 병렬로 시작하고 순차로 사용
async function hybrid() {
  // 먼저 모두 시작
  const userPromise = fetchUser(1);
  const productsPromise = fetchProducts();

  // 필요할 때 await
  const user = await userPromise;
  console.log(user);

  const products = await productsPromise;
  console.log(products);
}

```

await는 순차 실행

여러 await를 연속으로 작성하면 순차 실행됩니다. 독립적인 작업은 `Promise.all()` 로 병렬 실행하세요.

반복문에서의 async/await

```

const userIds = [1, 2, 3, 4, 5];

// 순차 실행 (하나씩)
async function sequential() {
  for (const id of userIds) {
    const user = await fetchUser(id);
    console.log(user);
  }
}

// 병렬 실행 (모두 동시에)
async function parallel() {
  const users = await Promise.all(
    userIds.map(id => fetchUser(id))
  );
  users.forEach(user => console.log(user));
}

// 주의: forEach는 await를 기다리지 않음!
async function wrong() {
  userIds.forEach(async (id) => {
    const user = await fetchUser(id);
    console.log(user);
  });
  console.log("완료"); // 모든 fetch 전에 출력됨!
}

```

Java CompletableFuture와 비교

JAVA

```
// 비동기 작업 생성
CompletableFuture<String> future =
    CompletableFuture.supplyAsync(() -> {
        return "결과";
    });

// 체이닝
future
    .thenApply(s -> s.toUpperCase())
    .thenAccept(System.out::println)
    .exceptionally(ex -> {
        System.err.println(ex);
        return null;
    });

// 여러 작업 병렬
CompletableFuture.allOf(f1, f2, f3)
    .thenRun(() -> {...});
```

JAVASCRIPT

```
// Promise 생성
const promise = new Promise(resolve => {
    resolve("결과");
});

// 체이닝
promise
    .then(s => s.toUpperCase())
    .then(console.log)
    .catch(console.error);

// 여러 작업 병렬
Promise.all([p1, p2, p3])
    .then(() => {...});

// async/await (더 권장)
const result = await promise;
```

기능	Java CompletableFuture	JavaScript
비동기 실행	supplyAsync()	new Promise()
변환	thenApply()	then()
소비	thenAccept()	then()
에러 처리	exceptionally()	catch()
병렬	allOf()	Promise.all()
경쟁	anyOf()	Promise.race()
동기 대기	get(), join()	await (async 함수 내)

실전 패턴

Fetch API

```
// 기본 GET 요청
const response = await fetch('/api/users');
const users = await response.json();

// POST 요청
const response = await fetch('/api/users', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({ name: '홍길동', age: 30 })
});

// 에러 체크 (fetch는 HTTP 에러를 reject하지 않음!)
async function fetchWithErrorHandling(url) {
  const response = await fetch(url);

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  return response.json();
}
```

타임아웃 구현

```

function timeout(ms) {
  return new Promise( (_, reject) => {
    setTimeout(() => reject(new Error('Timeout')), ms);
  });
}

// Promise.race로 타임아웃 구현
async function fetchWithTimeout(url, ms = 5000) {
  return Promise.race([
    fetch(url),
    timeout(ms)
  ]);
}

try {
  const response = await fetchWithTimeout('/api/slow', 3000);
} catch (error) {
  if (error.message === 'Timeout') {
    console.log('요청 시간 초과');
  }
}

```

재시도 로직

```

async function fetchWithRetry(url, maxRetries = 3) {
  for (let i = 0; i < maxRetries; i++) {
    try {
      const response = await fetch(url);
      if (response.ok) {
        return response.json();
      }
    } catch (error) {
      console.log(`시도 ${i + 1} 실패`);

      if (i === maxRetries - 1) {
        throw error;
      }

      // 지수 백오프
      await new Promise(r => setTimeout(r, 1000 * (i + 1)));
    }
  }
}

```

동시성 제한

```
// 한 번에 최대 3개씩만 실행
async function limitConcurrency(tasks, limit = 3) {
  const results = [];
  const executing = [];

  for (const task of tasks) {
    const p = task().then(result => {
      executing.splice(executing.indexOf(p), 1);
      return result;
    });

    results.push(p);
    executing.push(p);

    if (executing.length >= limit) {
      await Promise.race(executing);
    }
  }

  return Promise.all(results);
}

// 사용 예
const urls = ['/api/1', '/api/2', '/api/3', '/api/4', '/api/5'];
const tasks = urls.map(url => () => fetch(url));
const results = await limitConcurrency(tasks, 2);
```

Top-level await

ES2022부터 모듈의 최상위 레벨에서 await를 사용할 수 있습니다.

```
// config.js (ES 모듈)
const response = await fetch('/api/config');
export const config = await response.json();

// main.js
import { config } from './config.js';
// config가 로드된 후에 이 코드가 실행됨
console.log(config.apiUrl);
```

정리

JavaScript는 싱글스레드, 이벤트 루프로 비동기 처리

콜백: 기본 패턴이지만 중첩되면 읽기 어려움 (콜백 지옥)

Promise: `then`, `catch`, `finally` 로 체이닝

`Promise.all()`: 모두 완료 대기, `Promise.race()`: 가장 빠른 것

async/await: 가장 권장되는 방식, 동기 코드처럼 작성

여러 await는 순차 실행 → 독립적인 작업은 `Promise.all()` 로 병렬화

에러 처리: try/catch 또는 `.catch()`

fetch는 HTTP 에러를 reject하지 않으므로 `response.ok` 확인 필요

DOM과 이벤트

브라우저에서 JavaScript의 핵심은 DOM 조작입니다. 요소 선택, 조작, 이벤트 처리를 배우고 간단한 TODO 앱을 만들어봅시다.

DOM이란?

DOM(Document Object Model)은 HTML 문서를 JavaScript 객체로 표현한 것입니다. 브라우저가 HTML을 파싱하여 트리 구조의 객체로 만들어줍니다.

```
<!DOCTYPE html>
<html>
  <head>
    <title>제목</title>
  </head>
  <body>
    <h1 id="title">안녕하세요</h1>
    <p class="content">본문입니다</p>
  </body>
</html>
```

```
// DOM 트리 구조
document
├─ html
│   ├─ head
│   │   └─ title
│   │       └─ "제목" (텍스트 노드)
│   └─ body
│       ├─ h1#title
│       │   └─ "안녕하세요"
│       └─ p.content
│           └─ "본문입니다"
```

Java 개발자를 위한 비유

DOM은 Java의 XML DOM API와 유사합니다. Document, Element, Node 인터페이스가 있고, 트리 구조를 탐색하며 조작합니다.

요소 선택

단일 요소 선택

```
// ID로 선택 (가장 빠름)
const title = document.getElementById('title');

// CSS 선택자로 선택 (첫 번째 요소만)
const content = document.querySelector('.content');
const firstLink = document.querySelector('a');
const navLink = document.querySelector('nav > a.active');

// 예전 방식 (권장하지 않음)
document.getElementsByClassName('content')[0];
document.getElementsByTagName('p')[0];
```

여러 요소 선택

```

// CSS 선택자로 모든 요소 선택
const items = document.querySelectorAll('.item');
const links = document.querySelectorAll('a');

// NodeList 반환 (배열 비슷하지만 배열은 아님)
items.length;           // 요소 개수
items[0];               // 첫 번째 요소
items.forEach(...);     // forEach는 사용 가능

// 배열로 변환
const itemsArray = [...items];
// 또는
const itemsArray = Array.from(items);

// 예전 방식 (HTMLCollection 반환)
document.getElementsByClassName('item'); // 라이브 컬렉션
document.getElementsByTagName('a');

```

NodeList vs HTMLCollection

`querySelectorAll` 은 정적 NodeList를 반환합니다. `getElementsBy*` 는 라이브 HTMLCollection을 반환하여 DOM 변경 시 자동 업데이트됩니다. 일반적으로 `querySelector` / `querySelectorAll` 을 권장합니다.

탐색

```

const el = document.querySelector('.item');

// 부모
el.parentElement;
el.parentNode;

// 자식
el.children;           // HTMLCollection (요소만)
el.childNodes;         // NodeList (텍스트 노드 포함)
el.firstElementChild;
el.lastElementChild;

// 형제
el.previousElementSibling;
el.nextElementSibling;

// 가장 가까운 조상 찾기
el.closest('.container'); // CSS 선택자로 상위 요소 검색

```

요소 조작

내용 변경

```
const el = document.querySelector('#title');

// 텍스트 내용
el.textContent = '새 제목';    // HTML 태그는 텍스트로 표시
el.innerText = '새 제목';     // 화면에 보이는 텍스트만 (느낌)

// HTML 내용
el.innerHTML = '<strong>강조</strong> 텍스트';

// 외부 HTML (자기 자신 포함)
el.outerHTML = '<h2>대체됨</h2>';
```

innerHTML 보안 주의

사용자 입력을 `innerHTML` 에 직접 넣으면 XSS 공격에 취약합니다. 사용자 입력은 `textContent` 를 사용하거나 이스케이프 처리하세요.

속성 조작

```
const link = document.querySelector('a');

// 속성 읽기/쓰기
link.getAttribute('href');
link.setAttribute('href', 'https://example.com');
link.removeAttribute('target');
link.hasAttribute('rel');

// 직접 프로퍼티 접근 (표준 속성만)
link.href = 'https://example.com';
link.id = 'main-link';
link.className = 'btn primary';

// data 속성
// <div data-user-id="123" data-role="admin">
const el = document.querySelector('div');
el.dataset.userId;    // "123"
el.dataset.role;      // "admin"
el.dataset.newAttr = 'value'; // data-new-attr="value" 추가
```

클래스 조작

```
const el = document.querySelector('.box');

// classList API (권장)
el.classList.add('active');
el.classList.remove('hidden');
el.classList.toggle('open'); // 있으면 제거, 없으면 추가
el.classList.toggle('open', true); // 강제로 추가
el.classList.contains('active'); // true/false
el.classList.replace('old', 'new');

// 여러 클래스 한 번에
el.classList.add('a', 'b', 'c');
el.classList.remove('a', 'b');

// 예전 방식 (권장하지 않음)
el.className = 'box active';
el.className += ' new-class';
```

스타일 조작

```
const el = document.querySelector('.box');

// 인라인 스타일 설정
el.style.color = 'red';
el.style.backgroundColor = 'blue'; // camelCase
el.style.fontSize = '16px';
el.style.cssText = 'color: red; font-size: 16px;';

// 스타일 읽기
el.style.color; // 인라인 스타일만
getComputedStyle(el).color; // 실제 적용된 스타일

// 커스텀 속성 (CSS 변수)
el.style.setProperty('--main-color', 'blue');
getComputedStyle(el).getPropertyValue('--main-color');
```

요소 생성과 삭제

요소 생성

```
// 요소 생성
const div = document.createElement('div');
div.className = 'card';
div.textContent = '새 카드';

// 속성 설정
div.id = 'card-1';
div.dataset.index = '0';

// 자식 추가
const span = document.createElement('span');
span.textContent = '태그';
div.appendChild(span);

// DOM에 추가
document.body.appendChild(div);

// 특정 위치에 추가
const container = document.querySelector('.container');
container.appendChild(div);           // 끝에 추가
container.prepend(div);               // 앞에 추가
container.append(div, span, '텍스트'); // 여러 개 추가 가능

// 형제로 추가
const ref = document.querySelector('.reference');
ref.before(div);                      // 앞에
ref.after(div);                       // 뒤에

// 특정 위치에 삽입
container.insertBefore(div, ref);     // ref 앞에 삽입
```

요소 삭제

```

const el = document.querySelector('.item');

// 직접 삭제 (권장)
el.remove();

// 부모에서 삭제 (예전 방식)
el.parentElement.removeChild(el);

// 모든 자식 삭제
container.innerHTML = '';
// 또는
while (container.firstChild) {
  container.removeChild(container.firstChild);
}
// 또는
container.replaceChildren();

```

요소 복제

```

const original = document.querySelector('.template');

// 얕은 복제 (자식 제외)
const shallow = original.cloneNode(false);

// 깊은 복제 (자식 포함)
const deep = original.cloneNode(true);

document.body.appendChild(deep);

```

이벤트

이벤트 리스너

```
const button = document.querySelector('button');

// 이벤트 리스너 추가
button.addEventListener('click', function(event) {
    console.log('클릭됨!');
    console.log(event.target); // 클릭된 요소
});

// 화살표 함수 사용
button.addEventListener('click', (e) => {
    console.log('클릭됨!');
});

// 이름 있는 함수 (제거할 때 필요)
function handleClick(e) {
    console.log('클릭됨!');
}
button.addEventListener('click', handleClick);

// 이벤트 리스너 제거
button.removeEventListener('click', handleClick);

// 한 번만 실행
button.addEventListener('click', () => {
    console.log('한 번만');
}, { once: true });
```

주요 이벤트 종류

분류	이벤트	설명
마우스	click, dblclick, mousedown, mouseup, mousemove, mouseenter, mouseleave	클릭, 더블클릭, 마우스 이동 등
키보드	keydown, keyup, keypress	키 누름, 떼
폼	submit, change, input, focus, blur	폼 제출, 값 변경, 포커스
문서	DOMContentLoaded, load, scroll, resize	로드 완료, 스크롤, 크기 변경

이벤트 객체

```

element.addEventListener('click', (event) => {
    // 공통 속성
    event.type;           // 'click'
    event.target;         // 이벤트 발생 요소
    event.currentTarget;  // 리스너가 등록된 요소
    event.timeStamp;      // 이벤트 발생 시간

    // 마우스 이벤트
    event.clientX;        // 뷰포트 기준 X 좌표
    event.clientY;        // 뷰포트 기준 Y 좌표
    event.pageX;          // 문서 기준 X 좌표
    event.pageY;          // 문서 기준 Y 좌표
    event.button;         // 0: 왼쪽, 1: 가운데, 2: 오른쪽

    // 키보드 이벤트
    event.key;            // 'Enter', 'a', 'Shift' 등
    event.code;           // 'Enter', 'KeyA', 'ShiftLeft' 등
    event.ctrlKey;        // Ctrl 키 눌림 여부
    event.shiftKey;       // Shift 키 눌림 여부
    event.altKey;         // Alt 키 눌림 여부
    event.metaKey;        // Meta(Cmd) 키 눌림 여부

    // 기본 동작 방지
    event.preventDefault();

    // 이벤트 전파 중단
    event.stopPropagation();
});

```

이벤트 버블링과 캡처링

이벤트는 DOM 트리를 따라 전파됩니다. Java의 이벤트 시스템과 다른 개념이므로 주의가 필요합니다.

```

<div id="outer">
  <div id="inner">
    <button id="btn">클릭</button>
  </div>
</div>

```

이벤트 전파 단계

캡처링 (Capturing): document → html → body → outer → inner → button

타겟 (Target): button (실제 이벤트 발생)

버블링 (Bubbling): button → inner → outer → body → html → document

```
const outer = document.getElementById('outer');
const inner = document.getElementById('inner');
const btn = document.getElementById('btn');

// 기본값: 버블링 단계에서 실행
outer.addEventListener('click', () => console.log('outer'));
inner.addEventListener('click', () => console.log('inner'));
btn.addEventListener('click', () => console.log('btn'));

// 버튼 클릭 시 출력: btn → inner → outer

// 캡처링 단계에서 실행
outer.addEventListener('click', () => console.log('outer capture'), true);
// 또는 { capture: true }

// 버튼 클릭 시 출력: outer capture → btn → inner → outer
```

이벤트 전파 제어

```
// 버블링 중단
btn.addEventListener('click', (e) => {
  e.stopPropagation(); // 상위 요소로 전파 안 됨
  console.log('btn만 실행');
});

// 같은 요소의 다른 리스너도 중단
btn.addEventListener('click', (e) => {
  e.stopImmediatePropagation();
});

// 기본 동작 방지 (전파와는 별개)
const link = document.querySelector('a');
link.addEventListener('click', (e) => {
  e.preventDefault(); // 링크 이동 방지
});
```

이벤트 위임 (Event Delegation)

여러 자식 요소의 이벤트를 부모에서 한 번에 처리하는 패턴입니다. 동적으로 추가되는 요소에도 자동으로 적용됩니다.

```
<ul id="list">
  <li>항목 1</li>
  <li>항목 2</li>
  <li>항목 3</li>
</ul>

// 나쁜 예: 각 li에 리스너 등록
document.querySelectorAll('li').forEach(li => {
  li.addEventListener('click', () => console.log(li.textContent));
});

// 좋은 예: 부모에서 위임
const list = document.getElementById('list');
list.addEventListener('click', (e) => {
  // 클릭된 요소가 li인지 확인
  if (e.target.tagName === 'LI') {
    console.log(e.target.textContent);
  }

  // 또는 closest 사용
  const li = e.target.closest('li');
  if (li && list.contains(li)) {
    console.log(li.textContent);
  }
});

// 새로 추가된 항목에도 자동 적용
const newItem = document.createElement('li');
newItem.textContent = '항목 4';
list.appendChild(newItem); // 클릭하면 동작함
```

이벤트 위임의 장점

1. 메모리 절약 (리스너 하나만 등록)
2. 동적 요소 자동 처리
3. 코드 단순화

React, Vue 등 프레임워크도 내부적으로 이벤트 위임을 사용합니다.

폼 처리

```
<form id="myForm">
  <input type="text" name="username" id="username">
  <input type="email" name="email" id="email">
  <select name="role">
    <option value="user">사용자</option>
    <option value="admin">관리자</option>
  </select>
  <button type="submit">제출</button>
</form>

const form = document.getElementById('myForm');

// 폼 제출 이벤트
form.addEventListener('submit', (e) => {
  e.preventDefault(); // 기본 제출 동작 방지

  // 값 읽기
  const username = document.getElementById('username').value;
  const email = form.elements.email.value;
  const role = form.elements.role.value;

  // FormData 사용
  const formData = new FormData(form);
  const data = Object.fromEntries(formData);
  // { username: '...', email: '...', role: '...' }

  console.log(data);
});

// 입력 이벤트
const usernameInput = document.getElementById('username');

usernameInput.addEventListener('input', (e) => {
  console.log('현재 값:', e.target.value); // 입력할 때마다
});

usernameInput.addEventListener('change', (e) => {
  console.log('변경됨:', e.target.value); // 포커스 떠날 때
});

usernameInput.addEventListener('focus', () => console.log('포커스'));
usernameInput.addEventListener('blur', () => console.log('블러'));
```

실습: TODO 앱

지금까지 배운 내용을 활용하여 간단한 TODO 앱을 만들어봅시다.

```

<!DOCTYPE html>
<html>
<head>
  <title>TODO 앱</title>
  <style>
    .completed { text-decoration: line-through; color: gray; }
    .todo-item { padding: 8px; border-bottom: 1px solid #eee; }
    .delete-btn { color: red; cursor: pointer; margin-left: 10px; }
  </style>
</head>
<body>
  <h1>TODO 목록</h1>

  <form id="todoForm">
    <input type="text" id="todoInput" placeholder="할 일 입력">
    <button type="submit">추가</button>
  </form>

  <ul id="todoList"></ul>

  <script>
const form = document.getElementById('todoForm');
const input = document.getElementById('todoInput');
const list = document.getElementById('todoList');

// TODO 추가
form.addEventListener('submit', (e) => {
  e.preventDefault();

  const text = input.value.trim();
  if (!text) return;

  const li = document.createElement('li');
  li.className = 'todo-item';
  li.innerHTML = `
    <span class="todo-text">${escapeHtml(text)}</span>
    <span class="delete-btn">삭제</span>
  `;

  list.appendChild(li);
  input.value = '';
  input.focus();
});

// 이벤트 위임으로 완료/삭제 처리
list.addEventListener('click', (e) => {
  const li = e.target.closest('.todo-item');

```

```

    if (!li) return;

    // 삭제 버튼 클릭
    if (e.target.classList.contains('delete-btn')) {
        li.remove();
        return;
    }

    // 항목 클릭 시 완료 토글
    const text = li.querySelector('.todo-text');
    text.classList.toggle('completed');
});

// XSS 방지
function escapeHtml(text) {
    const div = document.createElement('div');
    div.textContent = text;
    return div.innerHTML;
}
</script>
</body>
</html>

```

TODO 앱에서 배운 것

1. 폼 제출 이벤트와 `preventDefault()`
2. `createElement()` 로 동적 요소 생성
3. 이벤트 위임으로 동적 요소 이벤트 처리
4. `classList.toggle()` 로 상태 전환
5. XSS 방지를 위한 텍스트 이스케이프

DOMContentLoaded vs load

```
// DOM 파싱 완료 시 (이미지 등 리소스 로드 전)
document.addEventListener('DOMContentLoaded', () => {
  console.log('DOM 준비됨');
});

// 모든 리소스(이미지, 스타일시트 등) 로드 완료 시
window.addEventListener('load', () => {
  console.log('모든 리소스 로드됨');
});

// 페이지 떠날 때
window.addEventListener('beforeunload', (e) => {
  e.preventDefault();
  e.returnValue = ''; // 확인 대화상자 표시
});
```

정리

DOM: HTML 문서를 JavaScript 객체 트리로 표현

선택: `querySelector`, `querySelectorAll` (CSS 선택자)

조작: `textContent`, `innerHTML`, `classList`, `style`

생성/삭제: `createElement`, `appendChild`, `remove`

이벤트: `addEventListener`, `removeEventListener`

전파: 캡처링 → 타겟 → 버블링

`stopPropagation`: 전파 중단, `preventDefault`: 기본 동작 방지

이벤트 위임: 부모에서 자식 이벤트 처리 (동적 요소에 유용)

보안: 사용자 입력은 `textContent` 사용 또는 이스케이프

모던 JavaScript 생태계

실무 JavaScript 개발에는 다양한 도구가 필요합니다. 패키지 관리자, 번들러, TypeScript를 소개하고 프레임워크 로드맵을 안내합니다.

npm - 패키지 관리자

npm(Node Package Manager)은 JavaScript의 패키지 관리자입니다. Java의 Maven/Gradle과 비슷한 역할을 합니다.

JAVA (Maven)

```
<!-- pom.xml -->
<dependencies>
  <dependency>
    <groupId>com.google.guava</groupId>
    <artifactId>guava</artifactId>
    <version>31.1-jre</version>
  </dependency>
</dependencies>

mvn install
```

JAVASCRIPT (npm)

```
// package.json
{
  "dependencies": {
    "lodash": "^4.17.21"
  }
}

npm install
// 또는
npm install lodash
```

프로젝트 시작

```
# 프로젝트 초기화
npm init
npm init -y # 기본값으로 빠르게 생성

# package.json 생성됨
{
  "name": "my-project",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

패키지 설치

```
# 프로덕션 의존성 (dependencies)
npm install lodash
npm i axios          # i는 install의 축약

# 개발 의존성 (devDependencies)
npm install --save-dev jest
npm i -D typescript

# 전역 설치
npm install -g typescript

# 특정 버전 설치
npm install lodash@4.17.21

# package.json의 모든 의존성 설치
npm install
npm ci # CI 환경에서 권장 (lock 파일 기준)
```

package.json

```

{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": {
    "start": "node src/index.js",
    "dev": "nodemon src/index.js",
    "build": "webpack --mode production",
    "test": "jest",
    "lint": "eslint src/"
  },
  "dependencies": {
    "express": "^4.18.2",
    "lodash": "^4.17.21"
  },
  "devDependencies": {
    "jest": "^29.5.0",
    "nodemon": "^2.0.22",
    "typescript": "^5.0.4"
  }
}

# 스크립트 실행
npm run start
npm start          # start, test, stop은 run 생략 가능
npm run build
npm test

```

버전 표기법

표기	의미	예시
^4.17.21	메이저 버전 고정	4.17.21 ~ 4.x.x
~4.17.21	마이너 버전 고정	4.17.21 ~ 4.17.x
4.17.21	정확한 버전	4.17.21만
*	최신 버전	모든 버전

npm 대안: yarn, pnpm

```
# yarn (Facebook)
yarn add lodash
yarn add -D typescript
yarn install

# pnpm (빠르고 디스크 효율적)
pnpm add lodash
pnpm add -D typescript
pnpm install
```

번들러

번들러는 여러 JavaScript 파일과 의존성을 하나(또는 최적화된 여러 개)의 파일로 묶어줍니다. 브라우저에서 효율적으로 로드할 수 있게 해줍니다.

번들러가 필요한 이유

1. 여러 파일을 하나로 합쳐 HTTP 요청 감소
2. 사용하지 않는 코드 제거 (tree-shaking)
3. 코드 압축 (minification)
4. 모듈 시스템 지원 (브라우저 호환성)

Webpack

가장 널리 사용되는 번들러입니다. 설정이 복잡하지만 강력합니다.

```
# 설치
npm install -D webpack webpack-cli

# webpack.config.js
const path = require('path');

module.exports = {
  entry: './src/index.js',
  output: {
    filename: 'bundle.js',
    path: path.resolve(__dirname, 'dist')
  },
  mode: 'production', // 또는 'development'
  module: {
    rules: [
      {
        test: /\.js$/,
        exclude: /node_modules/,
        use: 'babel-loader'
      },
      {
        test: /\.css$/,
        use: ['style-loader', 'css-loader']
      }
    ]
  }
};

# 빌드
npx webpack
npm run build # package.json scripts에 등록한 경우
```

Vite

최신 번들러로, 개발 서버가 매우 빠릅니다. 새 프로젝트에서 권장됩니다.

```
# 프로젝트 생성
npm create vite@latest my-app
cd my-app
npm install

# 개발 서버 시작
npm run dev

# 프로덕션 빌드
npm run build

# vite.config.js
import { defineConfig } from 'vite';

export default defineConfig({
  server: {
    port: 3000
  },
  build: {
    outDir: 'dist'
  }
});
```

WEBPACK

장점:

- 풍부한 생태계
- 레거시 브라우저 지원
- 세밀한 제어 가능

단점:

- 설정이 복잡
- 개발 서버가 느림
- 번들 크기가 커질 수 있음

VITE

장점:

- 설정이 간단
- 개발 서버가 매우 빠름
- ES 모듈 네이티브 활용

단점:

- 상대적으로 새로움
- 레거시 브라우저 지원 제한
- 플러그인 생태계가 작음

Babel - 트랜스파일러

최신 JavaScript 문법을 구형 브라우저에서도 동작하도록 변환합니다. Java의 컴파일러가 소스 코드를 바이트코드로 변환하는 것과 유사합니다.

```
# 설치
npm install -D @babel/core @babel/preset-env

# babel.config.json
{
  "presets": [
    ["@babel/preset-env", {
      "targets": "> 0.25%, not dead"
    }]
  ]
}

# 변환 예시 (입력)
const greet = (name = 'World') => `Hello, ${name}!`;
const [a, ...rest] = [1, 2, 3, 4];

# 변환 결과 (출력)
"use strict";
var greet = function greet() {
  var name = arguments.length > 0 ? arguments[0] : 'World';
  return "Hello, ".concat(name, "!");
};
var a = 1;
var rest = [2, 3, 4];
```

요즘은 대부분의 브라우저가 ES6를 지원하므로, IE 11 지원이 필요한 경우가 아니라면 Babel 없이도 개발할 수 있습니다.

TypeScript

TypeScript는 JavaScript에 정적 타입을 추가한 언어입니다. Java 개발자에게 가장 익숙한 JavaScript 개발 환경을 제공합니다.

JAVA

```
public class User {  
    private String name;  
    private int age;  
  
    public User(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String greet() {  
        return "안녕, " + this.name;  
    }  
}  
  
User user = new User("홍길동", 30);
```

TYPESCRIPT

```
class User {
  private name: string;
  private age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }

  greet(): string {
    return `안녕, ${this.name}`;
  }
}

const user: User = new User("홍길동", 30);
```

TypeScript 시작하기

```
# 설치
npm install -D typescript

# 설정 파일 생성
npx tsc --init

# tsconfig.json
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "strict": true,
    "outDir": "./dist",
    "rootDir": "./src"
  },
  "include": ["src/**/*"]
}

# 컴파일
npx tsc
```

기본 타입

```
// 기본 타입
let name: string = "홍길동";
let age: number = 30;
let isActive: boolean = true;

// 배열
let numbers: number[] = [1, 2, 3];
let names: Array<string> = ["a", "b"];

// 객체
let user: { name: string; age: number } = {
  name: "홍길동",
  age: 30
};

// 함수
function add(a: number, b: number): number {
  return a + b;
}

const greet = (name: string): string => `Hello, ${name}`;

// 선택적 파라미터
function greet(name: string, greeting?: string): string {
  return `${greeting ?? "Hello"}, ${name}`;
}
```

인터페이스와 타입

```

// 인터페이스 (Java의 interface와 유사)
interface User {
    id: number;
    name: string;
    email?: string; // 선택적 프로퍼티
    readonly createdAt: Date; // 읽기 전용
}

// 확장
interface Admin extends User {
    role: string;
    permissions: string[];
}

// 타입 별칭
type ID = string | number;
type Status = "pending" | "active" | "inactive";

// 제네릭 (Java와 동일)
interface ApiResponse<T> {
    data: T;
    status: number;
    message: string;
}

const response: ApiResponse<User> = {
    data: { id: 1, name: "홍길동", createdAt: new Date() },
    status: 200,
    message: "Success"
};

```

타입 가드

```
// 유니온 타입
function printId(id: string | number): void {
  if (typeof id === "string") {
    console.log(id.toUpperCase()); // string으로 추론
  } else {
    console.log(id.toFixed(2));    // number로 추론
  }
}

// instanceof
function processValue(value: Date | string): void {
  if (value instanceof Date) {
    console.log(value.toISOString());
  } else {
    console.log(value);
  }
}

// 사용자 정의 타입 가드
interface Dog { bark(): void; }
interface Cat { meow(): void; }

function isDog(animal: Dog | Cat): animal is Dog {
  return (animal as Dog).bark !== undefined;
}
```

TypeScript의 장점

1. 컴파일 타임 타입 검사로 버그 사전 방지
2. IDE 자동완성, 리팩토링 지원 향상
3. 코드 문서화 효과
4. Java 개발자에게 익숙한 문법
5. 대규모 프로젝트 유지보수 용이

린터와 포매터

ESLint - 린터

코드 품질과 스타일을 검사합니다. Java의 CheckStyle, SpotBugs와 유사합니다.

```
# 설치
npm install -D eslint

# 설정 초기화
npx eslint --init

# eslint.config.js (Flat Config)
export default [
  {
    rules: {
      "no-unused-vars": "warn",
      "no-console": "off",
      "eqeqeq": "error"
    }
  }
];

# 실행
npx eslint src/
npx eslint src/ --fix # 자동 수정
```

Prettier - 포맷터

코드 스타일을 일관되게 포맷합니다.

```
# 설치
npm install -D prettier

# .prettierrc
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5"
}

# 실행
npx prettier --write src/
```

프론트엔드 프레임워크

JavaScript를 어느 정도 익혔다면, 프론트엔드 프레임워크를 배울 준비가 된 것입니다.

React

Facebook이 개발한 가장 인기 있는 UI 라이브러리입니다. 컴포넌트 기반 개발과 가상 DOM을 사용합니다.

```
# 프로젝트 생성
npm create vite@latest my-app -- --template react
# TypeScript 버전
npm create vite@latest my-app -- --template react-ts

// 컴포넌트 예시
function Welcome({ name }) {
  return <h1>안녕하세요, {name}님!</h1>;
}

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <Welcome name="홍길동" />
      <button onClick={() => setCount(count + 1)}>
        클릭 수: {count}
      </button>
    </div>
  );
}
```

Vue.js

진입 장벽이 낮고 문서가 잘 되어 있는 프레임워크입니다. HTML 템플릿 문법이 직관적입니다.

```
# 프로젝트 생성
npm create vue@latest

// 컴포넌트 예시 (Composition API)
<script setup>
import { ref } from 'vue';

const count = ref(0);
const increment = () => count.value++;
</script>

<template>
  <h1>안녕하세요, {{ name }}님!</h1>
  <button @click="increment">
    클릭 수: {{ count }}
  </button>
</template>
```

Angular

Google이 개발한 풀 프레임워크입니다. TypeScript를 기본으로 사용하며, Java/Spring 개발자에게 익숙한 구조입니다.

```
# 프로젝트 생성
npm install -g @angular/cli
ng new my-app

// 컴포넌트 예시
@Component({
  selector: 'app-counter',
  template: `
    <h1>안녕하세요, {{ name }}님!</h1>
    <button (click)="increment()">
      클릭 수: {{ count }}
    </button>
  `
})
export class CounterComponent {
  name = '홍길동';
  count = 0;

  increment(): void {
    this.count++;
  }
}
```

프레임워크 비교

특성	React	Vue	Angular
학습 곡선	중간	낮음	높음
타입스크립트	선택	선택	기본
생태계	가장 큼	성장 중	포괄적
주요 특징	유연함, JSX간결함, 템플릿 풀 프레임워크		
적합한 프로젝트 모든 규모	중소규모	대규모	엔터프라이즈

백엔드: Node.js

JavaScript로 서버 사이트 개발도 가능합니다. Java Spring과 비슷한 역할을 하는 프레임워크들이 있습니다.

Express.js

```

const express = require('express');
const app = express();

app.use(express.json());

// GET /api/users
app.get('/api/users', (req, res) => {
  res.json([{ id: 1, name: '홍길동' }]);
});

// POST /api/users
app.post('/api/users', (req, res) => {
  const user = req.body;
  res.status(201).json(user);
});

app.listen(3000, () => {
  console.log('서버 시작: http://localhost:3000');
});

```

NestJS

Angular에서 영감을 받은 Node.js 프레임워크로, Spring과 유사한 구조입니다.

```

// Java Spring과 매우 유사한 구조
@Controller('users')
export class UsersController {
  constructor(private userService: UsersService) {}

  @Get()
  findAll(): User[] {
    return this.userService.findAll();
  }

  @Post()
  create(@Body() createUserDto: CreateUserDto): User {
    return this.userService.create(createUserDto);
  }
}

```

학습 로드맵

JavaScript 기초: 이 책의 내용 (완료!)

TypeScript: 타입 시스템 학습

프론트엔드 프레임워크: React 또는 Vue 선택

상태 관리: Redux, Zustand, Pinia 등

테스팅: Jest, Vitest, Testing Library

빌드 도구: Vite, Webpack 심화

(선택) 백엔드: Node.js, Express, NestJS

(선택) 풀스택: Next.js, Nuxt.js

정리

npm: JavaScript 패키지 관리자 (Maven/Gradle과 유사)

package.json: 프로젝트 설정, 의존성, 스크립트 정의

번들러: Webpack(설정 복잡, 강력), Vite(빠름, 간단)

Babel: 최신 문법을 구형 브라우저용으로 변환

TypeScript: 정적 타입 추가, Java 개발자에게 권장

ESLint: 코드 품질 검사, Prettier: 코드 포매팅

프레임워크: React(유연), Vue(간결), Angular(풀스택)

백엔드: Express(간단), NestJS(Spring과 유사)

실전 프로젝트

Spring Boot API와 연동하는 간단한 SPA를 만들어봅니다. CORS, Fetch API를 실습하고, 앞으로의 학습 방향을 안내합니다.

프로젝트 개요

이 장에서는 Spring Boot 백엔드와 통신하는 TODO 앱을 만듭니다. Java 개발자로서 익숙한 백엔드에 JavaScript 프론트엔드를 연결하는 실전 경험을 쌓습니다.

아키텍처



Spring Boot API

먼저 간단한 Spring Boot REST API를 준비합니다.

Todo 엔티티

```
// Todo.java
@Entity
public class Todo {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private boolean completed;

    // 생성자, getter, setter 생략
}
```

REST 컨트롤러

```
// TodoController.java
@RestController
@RequestMapping("/api/todos")
@CrossOrigin(origins = "http://localhost:5500") // CORS 허용
public class TodoController {

    @Autowired
    private TodoRepository todoRepository;

    @GetMapping
    public List<Todo> getAllTodos() {
        return todoRepository.findAll();
    }

    @PostMapping
    public Todo createTodo(@RequestBody Todo todo) {
        return todoRepository.save(todo);
    }

    @PutMapping("/{id}")
    public Todo updateTodo(@PathVariable Long id, @RequestBody Todo todo) {
        Todo existing = todoRepository.findById(id)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
        existing.setTitle(todo.getTitle());
        existing.setCompleted(todo.isCompleted());
        return todoRepository.save(existing);
    }

    @DeleteMapping("/{id}")
    public void deleteTodo(@PathVariable Long id) {
        todoRepository.deleteById(id);
    }
}
```

CORS 이해하기

CORS(Cross-Origin Resource Sharing)는 브라우저 보안 정책입니다. 프론트엔드(localhost:5500)와 백엔드(localhost:8080)가 다른 포트에서 실행되면 CORS 오류가 발생합니다.

CORS 오류

Access to fetch at 'http://localhost:8080/api/todos' from origin 'http://localhost:5500' has been blocked by CORS policy

이 오류는 서버에서 해결해야 합니다. 클라이언트에서는 해결할 수 없습니다.

Spring Boot에서 CORS 설정

```
// 방법 1: 컨트롤러에 @CrossOrigin 어노테이션
@CrossOrigin(origins = "http://localhost:5500")
@RestController
public class TodoController { }

// 방법 2: 전역 설정
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:5500")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("*")
            .allowCredentials(true);
    }
}
```

Fetch API

JavaScript에서 HTTP 요청을 보내는 표준 API입니다. Java의 HttpClient와 비슷하지만 Promise 기반입니다.

기본 사용법

```
// GET 요청
const response = await fetch('http://localhost:8080/api/todos');
const todos = await response.json();

// POST 요청
const response = await fetch('http://localhost:8080/api/todos', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({ title: '새 할일', completed: false })
});
const newTodo = await response.json();

// PUT 요청
await fetch(`http://localhost:8080/api/todos/${id}`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ title: '수정됨', completed: true })
});

// DELETE 요청
await fetch(`http://localhost:8080/api/todos/${id}`, {
  method: 'DELETE'
});
```

에러 처리

```

async function fetchTodos() {
  try {
    const response = await fetch('/api/todos');

    // HTTP 에러 체크 (fetch는 네트워크 에러만 reject)
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }

    return await response.json();
  } catch (error) {
    if (error.name === 'TypeError') {
      // 네트워크 에러 (서버 다운 등)
      console.error('네트워크 오류:', error);
    } else {
      console.error('요청 실패:', error);
    }
    throw error;
  }
}

```

API 클라이언트 모듈

```

// api.js
const API_BASE = 'http://localhost:8080/api';

async function request(endpoint, options = {}) {
  const url = `${API_BASE}${endpoint}`;

  const config = {
    headers: {
      'Content-Type': 'application/json',
      ...options.headers
    },
    ...options
  };

  if (config.body && typeof config.body === 'object') {
    config.body = JSON.stringify(config.body);
  }

  const response = await fetch(url, config);

  if (!response.ok) {
    const error = new Error(`HTTP ${response.status}`);
    error.status = response.status;
    throw error;
  }

  // 204 No Content 등 빈 응답 처리
  const text = await response.text();
  return text ? JSON.parse(text) : null;
}

// API 메서드
export const todoApi = {
  getAll: () => request('/todos'),

  getById: (id) => request(`/todos/${id}`),

  create: (todo) => request('/todos', {
    method: 'POST',
    body: todo
  }),

  update: (id, todo) => request(`/todos/${id}`, {
    method: 'PUT',
    body: todo
  }),

```

```
delete: (id) => request(`/todos/${id}`, {  
    method: 'DELETE'  
})  
};
```

SPA 구현

이제 Spring Boot API와 연동하는 완전한 SPA를 만들어봅시다.

HTML 구조

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TODO 앱</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      font-family: -apple-system, sans-serif;
      background: #f5f5f5;
      min-height: 100vh;
      padding: 40px 20px;
    }
    .container {
      max-width: 500px;
      margin: 0 auto;
      background: white;
      border-radius: 8px;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
      padding: 24px;
    }
    h1 { margin-bottom: 24px; color: #333; }
    .input-group {
      display: flex;
      gap: 8px;
      margin-bottom: 24px;
    }
    input[type="text"] {
      flex: 1;
      padding: 12px;
      border: 1px solid #ddd;
      border-radius: 4px;
      font-size: 16px;
    }
    button {
      padding: 12px 20px;
      background: #4CAF50;
      color: white;
      border: none;
      border-radius: 4px;
      cursor: pointer;
      font-size: 14px;
    }
    button:hover { background: #45a049; }
    button.delete {
      background: #f44336;
```

```

        padding: 6px 12px;
    }
    .todo-list { list-style: none; }
    .todo-item {
        display: flex;
        align-items: center;
        gap: 12px;
        padding: 12px;
        border-bottom: 1px solid #eee;
    }
    .todo-item:last-child { border-bottom: none; }
    .todo-item.completed span { text-decoration: line-through; color: #999; }
    .todo-item span { flex: 1; }
    .loading { text-align: center; color: #999; padding: 20px; }
    .error { color: #f44336; padding: 12px; background: #ffebee; border-radius: 4px; }
</style>
</head>
<body>
    <div class="container">
        <h1>TODO 목록</h1>

        <div class="input-group">
            <input type="text" id="todoInput" placeholder="할 일을 입력하세요">
            <button id="addBtn">추가</button>
        </div>

        <div id="error" class="error" style="display: none;"></div>
        <div id="loading" class="loading">로딩 중...</div>
        <ul id="todoList" class="todo-list"></ul>
    </div>

    <script type="module" src="app.js"></script>
</body>
</html>

```

JavaScript 모듈

```

// app.js
const API_BASE = 'http://localhost:8080/api/todos';

// 상태
let todos = [];

// DOM 요소
const todoInput = document.getElementById('todoInput');
const addBtn = document.getElementById('addBtn');
const todoList = document.getElementById('todoList');
const loadingEl = document.getElementById('loading');
const errorEl = document.getElementById('error');

// API 함수
async function fetchTodos() {
  const response = await fetch(API_BASE);
  if (!response.ok) throw new Error('조회 실패');
  return response.json();
}

async function createTodo(title) {
  const response = await fetch(API_BASE, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title, completed: false })
  });
  if (!response.ok) throw new Error('생성 실패');
  return response.json();
}

async function updateTodo(id, data) {
  const response = await fetch(`${API_BASE}/${id}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data)
  });
  if (!response.ok) throw new Error('수정 실패');
  return response.json();
}

async function deleteTodo(id) {
  const response = await fetch(`${API_BASE}/${id}`, {
    method: 'DELETE'
  });
  if (!response.ok) throw new Error('삭제 실패');
}

```

```

// UI 함수
function showError(message) {
  errorEl.textContent = message;
  errorEl.style.display = 'block';
  setTimeout(() => errorEl.style.display = 'none', 3000);
}

function showLoading(show) {
  loadingEl.style.display = show ? 'block' : 'none';
}

function render() {
  todoList.innerHTML = todos.map(todo => `
    <li class="todo-item ${todo.completed ? 'completed' : ''}" data-id="${todo.id}">
      <input type="checkbox" ${todo.completed ? 'checked' : ''}>
      <span>${escapeHtml(todo.title)}</span>
      <button class="delete">삭제</button>
    </li>
  `).join('');
}

function escapeHtml(text) {
  const div = document.createElement('div');
  div.textContent = text;
  return div.innerHTML;
}

// 이벤트 핸들러
async function handleAdd() {
  const title = todoInput.value.trim();
  if (!title) return;

  try {
    const newTodo = await createTodo(title);
    todos.push(newTodo);
    render();
    todoInput.value = '';
  } catch (error) {
    showError('추가 실패: ' + error.message);
  }
}

async function handleToggle(id) {
  const todo = todos.find(t => t.id === id);
  if (!todo) return;

  try {
    const updated = await updateTodo(id, {

```

```

        ...todo,
        completed: !todo.completed
    });
    todos = todos.map(t => t.id === id ? updated : t);
    render();
} catch (error) {
    showError('수정 실패: ' + error.message);
}
}

async function handleDelete(id) {
    try {
        await deleteTodo(id);
        todos = todos.filter(t => t.id !== id);
        render();
    } catch (error) {
        showError('삭제 실패: ' + error.message);
    }
}

// 이벤트 바인딩
addBtn.addEventListener('click', handleAdd);

todoInput.addEventListener('keypress', (e) => {
    if (e.key === 'Enter') handleAdd();
});

// 이벤트 위임
todoList.addEventListener('click', (e) => {
    const item = e.target.closest('.todo-item');
    if (!item) return;

    const id = Number(item.dataset.id);

    if (e.target.type === 'checkbox') {
        handleToggle(id);
    } else if (e.target.classList.contains('delete')) {
        handleDelete(id);
    }
});

// 초기화
async function init() {
    showLoading(true);
    try {
        todos = await fetchTodos();
        render();
    } catch (error) {

```

```

        showError('데이터를 불러올 수 없습니다. 서버를 확인하세요.');
```

```

    } finally {
        showLoading(false);
    }
}

init();
```

SPA 핵심 패턴

1.

상태(State)

: `todos` 배열로 데이터 관리

2.

렌더링

: 상태가 변경되면 `render()` 호출

3.

이벤트

: 사용자 액션 → API 호출 → 상태 업데이트 → 렌더링

4.

이벤트 위임

: 동적 요소는 부모에서 이벤트 처리

개발 환경 실행

```

# Spring Boot 실행 (8080 포트)
./mvnw spring-boot:run
# 또는
./gradlew bootRun

# 프론트엔드 실행 (VS Code Live Server 또는)
npx serve .
# 또는 Python
python -m http.server 5500
```

개선 포인트

로딩 상태 개선

```
// 개별 항목 로딩 표시
async function handleDelete(id) {
  const item = document.querySelector(`[data-id="${id}"]`);
  item.style.opacity = '0.5';

  try {
    await deleteTodo(id);
    todos = todos.filter(t => t.id !== id);
    render();
  } catch (error) {
    item.style.opacity = '1';
    showError('삭제 실패');
  }
}
```

낙관적 업데이트

```
// 즉시 UI 업데이트, 실패 시 롤백
async function handleToggle(id) {
  const todo = todos.find(t => t.id === id);
  const originalCompleted = todo.completed;

  // 낙관적 업데이트
  todo.completed = !todo.completed;
  render();

  try {
    await updateTodo(id, todo);
  } catch (error) {
    // 실패 시 롤백
    todo.completed = originalCompleted;
    render();
    showError('수정 실패');
  }
}
```

디바운싱

```
// 연속 입력 시 마지막 입력만 처리
function debounce(fn, delay) {
  let timeoutId;
  return function(...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => fn.apply(this, args), delay);
  };
}

const debouncedSearch = debounce(async (query) => {
  const results = await searchTodos(query);
  renderResults(results);
}, 300);

searchInput.addEventListener('input', (e) => {
  debouncedSearch(e.target.value);
});
```

다음 단계

이 책에서 배운 내용을 바탕으로 다음 단계로 나아갈 수 있습니다.

프론트엔드 심화

TypeScript: 타입 안전성 확보

React/Vue: 컴포넌트 기반 개발

상태 관리: Redux, Zustand, Pinia

라우팅: React Router, Vue Router

테스팅: Jest, Vitest, Testing Library

풀스택 개발

Next.js: React 풀스택 프레임워크

Nuxt.js: Vue 풀스택 프레임워크

Node.js: JavaScript 백엔드

NestJS: Spring과 유사한 Node.js 프레임워크

추천 학습 자료

MDN Web Docs: JavaScript 공식 문서

javascript.info: 체계적인 JavaScript 튜토리얼

TypeScript 공식 문서: Handbook

React 공식 문서: 새로운 beta 문서

Vue 공식 문서: 한국어 지원

축하합니다!

이 책을 완독하셨습니다. Java 개발자로서 JavaScript의 기초를 다졌습니다. 이제 TypeScript와 프론트엔드 프레임워크를 학습하여 풀스택 개발자로 성장하세요. JavaScript 생태계에서의 여정을 응원합니다!

정리

Spring Boot REST API와 JavaScript SPA 연동

CORS: 다른 출처 간 요청 허용 설정 (서버에서 처리)

Fetch API: HTTP 요청 (GET, POST, PUT, DELETE)

에러 처리: response.ok 확인, try/catch

SPA 패턴: 상태 관리 → 렌더링 → 이벤트 처리

이벤트 위임: 동적 요소 이벤트 처리

다음 단계: TypeScript, React/Vue, 풀스택